Discrete Event Traffic Simulation Framework using Object-Driven Cellular Automata

Kerem Demirtaş[a,*], Pitu Mirchandani[a], Xuesong Zhou[b]

[a]*School of Computing and Augmented Intelligence, Arizona State University, Tempe, AZ, 85287, USA*
[b]*School of Sustainable Engineering and the Built Environment, Arizona State University, Tempe, AZ, 85287, USA*

Abstract

Cellular automata have run on discrete-event engines since Cell-DEVS, but the cell remains active and the moving object a value in its state. This paper inverts that assignment for traffic microsimulation. In the Object-Driven Cellular Automata (ODCA) framework, each vehicle is an asynchronous process and each cell is a resource of capacity one, so behaviour attaches to the vehicle; the cell holds state but no transition rules. A vehicle acquires cells through a request–wait–seize–delay–release protocol, congestion emerges from resource contention rather than from cell rules or explicit gap checks, and speed is a real number rather than a discrete cell state, removing the integer-speed artifact of cell-based models, event-driven ones included. The delayed release produces headways consistent with Newell's car-following model, yielding a triangular fundamental diagram without calibration. Drivers re-evaluate speed and direction when neighbours move rather than at fixed intervals, so computation concentrates where vehicles interact. We derive closed-form expressions linking the protocol parameters to capacity, critical density and backward wave speed, and extend them to mixed autonomous and human-driven traffic. In 20 replications on a four-lane freeway offered 7,000 veh/h, raising the autonomous share from 0% to 70% increases served throughput by 18.7% (from $5{,}885 \pm 24$ to $6{,}983 \pm 35$ veh/h, 95% CI) and cuts average delay by 96% (from 215 ± 5 to 8.6 ± 0.3 s). Throughput understates the capacity effect once the network serves 99.8% of the demand offered, after which the benefit appears as delay reduction, at a lane-drop bottleneck as well.

Keywords: cellular automata, discrete event simulation, traffic microsimulation, car following, lane changing, autonomous vehicles, mixed traffic

1. Introduction

A spatially explicit simulation model has to decide where behaviour lives. In a cellular automaton, the lattice is active: every cell holds a state and a transition rule, and a moving object is a pattern of states that the rules propagate from cell to cell. The discrete-event formalisms built on that structure keep the same assignment. In Cell-DEVS [1, 2], and in its traffic applications [3, 4], each cell is an atomic DEVS model with its own transition functions, and moving to an event-driven engine buys asynchrony without changing which entity carries the behaviour. This paper inverts that assignment, makes the moving object the process and the cell a resource without rules of its own, and reports what the inversion costs and what it buys. Freeway traffic with mixed autonomous and human-driven

*Corresponding author
Email address: keremdemirtas@gmail.com (Kerem Demirtaş)

vehicles is the setting, because it is one in which the answer can be measured against theory rather than only demonstrated.

That setting is not a convenience. Autonomous vehicles (AVs) and human-driven vehicles (HDVs) differ in reaction time, headway preference and lane-change behavior, and a simulation framework has to represent both at the vehicle level while staying computationally tractable.

Existing simulation approaches fall broadly into two categories. Microscopic traffic simulators such as SUMO [5] and Vissim [6] model individual vehicles with detailed behavioral rules but rely on fixed-timestep updates: every vehicle is evaluated at every timestep (typically 0.1–1.0 s) regardless of whether any meaningful interaction has occurred. This synchronous update scheme introduces computational redundancy, particularly in free-flow regions where vehicles travel independently. Cellular automata (CA) models, introduced by Nagel and Schreckenberg [7], offer a computationally efficient alternative through discretized space and simple local update rules. However, classical CA models also employ synchronous parallel updates and restrict vehicle speeds to integer values (cells per timestep), producing artificial speed plateaus and unrealistic fundamental diagrams [8].

Both paradigms share a fundamental limitation: time advances in fixed increments, forcing all vehicles to synchronize at a global clock. This is at odds with the physical reality of traffic, where vehicle interactions are inherently asynchronous: a lane change on one side of the freeway has no immediate causal relationship with car following on the other. Moreover, the fixed-timestep approach makes it difficult to naturally model the heterogeneous perception and reaction characteristics of mixed AV/HDV traffic, where AVs may operate at update frequencies orders of magnitude higher than human reaction times.

This paper proposes an alternative paradigm: an **Object-Driven Cellular Automata (ODCA)** framework embedded in a **Discrete Event Simulation (DES)** environment. The key innovations are:

i) **Vehicle-centric dynamics: the cell has state but no rules.** Each vehicle is modeled as an autonomous simulation process that advances through the freeway by acquiring and releasing cell resources. Time progresses from event to event, not in fixed increments, so computation is concentrated where interactions actually occur. Putting a cellular automaton on a discrete-event engine is not itself new: Cell-DEVS [1, 2] has done so since 2001, and has been applied to traffic [3, 4]. What is new here is the inversion of which entity is active. In Cell-DEVS the cell is an atomic model holding state and a transition function, and a vehicle is a value that propagates between cells; in ODCA-DES the vehicle is the process, and the cell holds state (its occupancy, whether it is blocked, the speed limit posted on it) but no transition rule: it never changes its own state or another's, and every behaviour belongs to the vehicle. Section 2.3 states the consequences.

ii) **Resource-based conflict resolution.** Freeway cells are modeled as priority-allowing resources. A vehicle requesting an occupied cell simply waits until the resource is released, and congestion emerges naturally from resource contention rather than explicit gap-checking rules.

iii) **Hybrid discretization.** Space is discrete (cells), but vehicle speeds are continuous and time advances continuously through event scheduling. This eliminates the integer-speed constraint of classical CA while preserving the simple spatial structure that makes CA models tractable.

iv) **Native $T(x, n)$ trajectory output.** The framework operates in the passage-time coordinate system, producing trajectory data that directly records the time each vehicle passes each cell. This aligns with Newell's

kinematic wave theory [9] and enables analytical connections to traffic flow theory that timestep-based simulators cannot provide natively.

v) **Decoupled driver and movement architecture.** The behavioral layer (speed and direction decisions) is separated from the physical layer (cell-by-cell resource acquisition), enabling modular integration of car-following models, lane-change models, and AV control logic without modifying the movement engine.

vi) **Event-driven driver process.** Each vehicle's driver re-evaluates speed and lane-change decisions not at fixed intervals, but in response to movements of neighboring vehicles. This push-based reactive architecture means that vehicles in free flow generate minimal computational events, while vehicles in congested regions respond immediately to local density changes, a property that fixed-timestep simulators cannot provide without evaluating every vehicle at every step.

vii) **Physically motivated lane-change safety.** Lane-change feasibility is assessed through speed-dependent closing-speed gaps, and mandatory lane changes incorporate blockage-aware urgency inflation, producing realistic incident dynamics without cooperative communication between vehicles.

viii) **Closed-form capacity expression for mixed traffic.** The resource protocol parameters yield an analytical expression for the capacity effect of AV penetration, providing a direct link between simulation mechanics and macroscopic traffic quantities.

The result is an event-driven microsimulation on a cellular lattice. It combines the behavioral detail of microscopic models, the spatial simplicity of cellular automata and the asynchronous efficiency of discrete event simulation. The modelling pattern itself does not depend on traffic. A lattice of unit-capacity resources, acquired by object processes that carry all the behaviour, could serve wherever discrete space is a contended medium. This paper reports one domain only, so that transfer is a property of the protocol, not a demonstrated result; a second domain is future work. Traffic offers a theory to check the pattern against. Section 4 shows that the protocol's release delay reproduces Newell's headways and a triangular fundamental diagram in closed form, with no parameter fitted to produce them.

The remainder of this paper is organized as follows. Section 2 reviews the relevant literature on cellular automata, traffic microsimulation, and discrete event simulation in transportation. Section 3 presents the ODCA-DES framework in detail, including the hybrid discretization scheme, resource-based movement protocol, and decoupled driver architecture. Section 4 derives analytical properties of the framework, including headway consistency and fundamental diagram relationships. Section 5 presents computational experiments demonstrating the framework's capabilities. Section 6 concludes with a discussion of contributions and future research directions.

2. Literature Review

2.1. Cellular Automata Models of Traffic

The foundational CA traffic model was introduced by Nagel and Schreckenberg [7], known as the Nagel-Schreckenberg (NaSch) model. In this model, a single-lane road is discretized into cells, each either empty or occupied by a single vehicle. At each discrete timestep, all vehicles simultaneously execute four rules: acceleration, deceleration (due to the preceding vehicle), randomization (stochastic slowdown), and movement. Vehicle speeds

are restricted to non-negative integers bounded by a maximum value $v_{\max}$, representing the number of cells a vehicle advances per timestep.

Rickert et al. [10] extended the NaSch model to two lanes by introducing symmetric and asymmetric lane-change rules based on incentive and safety criteria. Subsequent extensions addressed slow-to-start behavior [11], velocity-dependent randomization, and anticipation effects. Maerivoet and De Moor [8] provide a comprehensive review of CA traffic models and their variants.

Despite their computational efficiency, CA models share several limitations that motivate the present work:

- **Synchronous updates:** All vehicles update simultaneously at each timestep. This imposes an artificial global synchronization that does not reflect the asynchronous nature of real driver behavior.
- **Integer speed constraint:** Speeds are restricted to $\{0, 1, \ldots, v_{\max}\}$ cells per timestep. This produces stepped fundamental diagrams and prevents accurate representation of car-following models that require continuous speed adjustments.
- **Cell-centric perspective:** The simulation iterates over cells (or a global vehicle list with identical update timing), rather than allowing each vehicle to evolve on its own timeline. This makes it difficult to model heterogeneous populations with different perception delays and update frequencies.
- **Rule-based interactions:** Vehicle interactions are governed by explicit conditional rules (e.g., "if gap $<$ speed, then decelerate") rather than emerging from physical resource constraints.

2.2. Microscopic Traffic Simulation

Microscopic simulators model individual vehicles with detailed behavioral sub-models for car following, lane changing, and route choice. Widely used platforms include SUMO [5], Vissim [6], and CORSIM. These simulators employ continuous-space vehicle tracking with high-fidelity behavioral models such as the Intelligent Driver Model [IDM; 12], Gipps' model [13], and the MOBIL lane-change model [14].

However, all major microsimulators operate on a fixed-timestep paradigm, typically with $\Delta t \in [0.1, 1.0]$ s. At each timestep, the simulator evaluates every vehicle's acceleration and lane-change decisions, regardless of whether the vehicle is in a meaningful interaction. This creates computational overhead proportional to $N \times T/\Delta t$ where N is the number of vehicles and T is the simulation duration, even when large portions of the network are in free flow.

2.3. Discrete Event Simulation in Transportation

Discrete event simulation (DES) is a modeling paradigm in which system state changes occur at discrete points in time triggered by events, and the simulation clock advances from event to event rather than in fixed increments [15]. DES is the dominant paradigm in manufacturing, logistics, healthcare, and queueing systems, where entities (customers, parts, patients) move through networks of resources (machines, servers, beds) via process-based interactions.

Despite its natural fit for modeling vehicles moving through infrastructure, DES has seen limited adoption in *microscopic* traffic simulation, where the fixed-timestep paradigm remains dominant. Burghout's hybrid mesoscopic-microscopic framework [16] uses DES at the mesoscopic level to model link traversal times while delegating vehicle interactions to a microscopic sub-model. Several studies have explored event-based traffic signal control [17], and queueing-network representations of signalized intersections naturally lend themselves to DES [18]. These approaches model vehicles at an aggregate or mesoscopic level, without the individual trajectory resolution needed for lane-level interactions, car-following dynamics, and mixed-autonomy analysis.

The closest antecedent of the present work is the Cell-DEVS formalism [1, 2], a discrete-event specification of cellular automata built on Zeigler's DEVS [19]. Cell-DEVS makes each cell an atomic DEVS model whose state changes propagate through the cell space via transport or inertial delays, so cell updates are asynchronous and event-driven rather than synchronous, and it has been applied to traffic directly: Davidson and Wainer [3] specify truck movement as Cell-DEVS rules, and the ATLAS language [4] compiles city sections into Cell-DEVS cell spaces for microsimulation. Combining a cell-based spatial structure with a discrete-event engine is therefore established ground, and the present work does not claim it.

ODCA-DES therefore shares with Cell-DEVS the rejection of the synchronous update, and departs from it on three points, of which the first entails the other two.

What is active. In Cell-DEVS the cell is the atomic model: it holds state, owns a transition function, and a vehicle exists as a value that propagates from one cell's state to the next. In ODCA-DES the vehicle is the process and the cell is a resource of capacity one. The cell still holds state, its occupancy, whether it is blocked and the speed limit posted on it, but it has no transition rule: vehicles read that state and change the occupancy, a scenario event may block or reopen the cell, and the cell itself never acts. Behavior is attached to the object that has it in the real system, which is why a heterogeneous fleet costs nothing structurally here: a driver model is a property of a vehicle process, not a case in a rule that every cell in the lattice must evaluate.

What a speed is. A Cell-DEVS traffic cell still encodes speed in its discrete state, so the integer-speed artifact of classical CA survives the move to discrete events. In ODCA-DES speed is a real number carried by the vehicle, and the cell crossing time ℓ/v is whatever that number makes it, so the artifact does not survive (Section 4.3.1).

Where congestion comes from. Cell-DEVS expresses interaction as cell rules over a neighborhood, in the CA tradition. ODCA-DES expresses it as contention for a resource: a vehicle requesting an occupied cell waits, and the delayed release of Section 3.5 makes that wait reproduce Newell's headway analytically rather than by rule calibration. This is the step that yields the closed-form capacity of Section 4.2, which has no counterpart in the Cell-DEVS traffic models.

To the authors' knowledge, this paper is the first to model individual vehicle movement as the acquisition of cells as resources, and the first to derive headway and capacity in closed form from that acquisition protocol.

The present framework adopts the DES paradigm through SimPy, a process-based DES library for Python [20]. In this paradigm, each vehicle is a generator process that can request resources, wait for availability, and proceed upon acquisition, a natural mapping to the request–seize–delay–release cycle of cell-by-cell vehicle movement.

2.4. Car-Following Models

Car-following models describe how a vehicle adjusts its speed in response to the vehicle directly ahead. Among the most influential are Gipps' safe-distance model [13], the Optimal Velocity Model [21], the Intelligent Driver Model [12], and Newell's simplified car-following model [22].

Newell's model is of particular relevance to this work. It posits that a follower's trajectory is a translation of the leader's trajectory, shifted by a time offset τ (reaction time) and a spatial offset d, the jam spacing: the distance from the front of one stopped vehicle to the front of the next, that is, a vehicle length plus the minimum gap between stopped vehicles:

$$X_n(t) = X_{n-1}(t - \tau) - d \tag{1}$$

In the passage-time coordinate, this becomes:

$$T(x, n) = T(x + d,\, n - 1) + \tau \tag{2}$$

which states that vehicle n passes position x exactly τ seconds after its leader passed position $x+d$. This formulation aligns directly with the ODCA-DES framework, where the delayed cell release mechanism produces an equivalent relationship (Section 3.5).

2.5. Lane-Changing Models

Lane-changing behavior is typically decomposed into mandatory lane changes (MLC), driven by route requirements such as reaching an exit, and discretionary lane changes (DLC), motivated by speed or comfort incentives [23]. Classical models include the rule-based approach of Gipps [24], the utility-based framework of Toledo et al. [25], and the minimizing-overall-braking-induced-by-lane-changes (MOBIL) model of Kesting et al. [14].

In the ODCA-DES framework, lane-change decisions are modeled using logistic probability functions [26] that capture the increasing urgency of mandatory lane changes as a vehicle approaches its destination exit. The driver process evaluates lane-change decisions asynchronously, decoupled from the cell-by-cell movement process, and lane changes are executed as diagonal resource acquisitions: the vehicle seizes the next cell in the adjacent lane. The detailed formulation is presented in Section 3.6.2.

2.6. Mixed Autonomous and Human-Driven Traffic

The coexistence of AVs and HDVs on freeways creates a heterogeneous traffic environment with distinct behavioral characteristics. AVs typically exhibit shorter reaction times, smaller desired headways, and more predictable lane-change behavior compared to HDVs [27, 28, 29]. Modeling this heterogeneity requires simulation frameworks that can accommodate diverse behavioral parameter sets within the same traffic stream.

The object-oriented architecture of ODCA-DES addresses this need directly: AV and HDV classes inherit from a common vehicle base class but carry different parameter sets (reaction time τ, randomization probability, lane-change aggressiveness) and can implement distinct behavioral models (e.g., cooperative lane changing for AVs vs. gap-acceptance for HDVs).

3. ODCA-DES Framework

This section presents the Object-Driven Cellular Automata framework within a Discrete Event Simulation environment. We describe the hybrid discretization scheme, infrastructure representation, vehicle modeling, and the decoupled driver–movement architecture.

3.1. Hybrid Discretization

The ODCA-DES framework employs a hybrid discretization scheme that combines discrete spatial representation with continuous speed and continuous time:

- **Space:** The freeway is discretized into cells of length ℓ (typically 7.5 m, an average vehicle length plus the minimum gap between stopped vehicles). The cell length is therefore the jam spacing of Section 2.4: $d = \ell = 7.5\,\text{m}$, and a stopped vehicle occupies exactly one cell. Each lane is a sequence of cells indexed by integer positions $c \in \{1, 2, \ldots, C\}$, where C is the number of cells per lane.
- **Speed:** Vehicle speeds are continuous-valued, $v^{(i)} \in [0, \hat{v}^{(i)}(c)]$, where $\hat{v}^{(i)}(c)$ is the active maximum speed of vehicle i on cell c (Eq. 10), determined by behavioral models (car following, free flow). This eliminates the integer-speed artifact of classical CA and enables smooth fundamental diagrams.
- **Time:** Simulation time is continuous, advanced by the DES event scheduler from event to event. There is no global Δt; the time between consecutive events for a given vehicle is ℓ/v, which varies continuously with speed.

This hybrid scheme preserves the spatial simplicity of CA (cells as discrete, addressable units) while removing the two most restrictive CA assumptions: integer speeds and synchronous timesteps. The discrete spatial structure provides a natural resource model (Section 3.4) and simplifies lane-change gap assessment to checking adjacent cell occupancy.

3.2. Infrastructure Model

The freeway infrastructure is modeled as a collection of interconnected objects:

- **Cell:** The fundamental spatial unit. Each cell is implemented as a DES *PriorityResource* with capacity one, meaning at most one vehicle can occupy (seize) the cell at any given time; requests waiting for the same cell are served in order of a priority value, first come first served among equal values (Section 3.6.2). Besides its occupancy, each cell carries a blocked flag (a blocked cell is closed to vehicles, which is how incidents and lane closures are represented) and a posted speed limit $v_{\text{lim}}(c)$. Cells maintain references to their longitudinal predecessor and successor, their lateral neighbors (cells in adjacent lanes at the same longitudinal position) and their diagonal neighbors (one cell ahead or behind in an adjacent lane).
- **Lane:** An ordered sequence of cells representing one lane of the freeway. Lanes are indexed from 1 (rightmost, nearest to ramps) to L (leftmost).

- **Segment:** A collection of parallel lanes sharing the same longitudinal extent, with associated speed limits and geometric properties.
- **Origin:** A named place where vehicles enter the road, joined to one road cell: the first cell of a lane at the start of the segment (the mainline entries) or the merge cell of an on-ramp. Vehicles generated for an origin queue there in order and enter when they can seize its road cell, so the road itself limits what enters.
- **On-ramp:** A source that generates vehicles and injects them into designated merge cells in the rightmost lane, subject to resource availability.
- **Off-ramp:** A sink connected to designated cells in the rightmost lane. Vehicles with matching destination exit the freeway by releasing their current cell and entering the off-ramp.
- **Segment end:** Vehicles whose destination is the end of the segment exit from their current lane upon reaching the final cell, without needing to merge to a specific lane.

3.3. Vehicle Model

Each vehicle i is an autonomous DES process characterized by:

- **Static attributes:** vehicle type (AV or HDV), origin, destination (target off-ramp), maximum speed $v_{\max}^{(i)}$, vehicle length.
- **Behavioral parameters:** reaction time $\tau^{(i)}$, random slowdown probability $p^{(i)}$ (HDV only), lane-change urgency parameters.
- **Dynamic state:** current cell $c^{(i)}(t)$, current speed $v^{(i)}(t)$, current lane $j^{(i)}(t)$, leader vehicle reference.
- **Trajectory record:** the sequence of tuples $\{(c,\, T^{\text{arr}}(c,i))\}$ recording the arrival time at each cell traversed.

The vehicle object encapsulates two concurrent processes: the *movement process* and the *driver process*, which operate asynchronously and interact through the vehicle's state variables.

3.4. Movement Process

The movement process governs the physical progression of a vehicle through the cellular lattice. For each cell transition, the vehicle executes the following protocol:

1. **Request:** Vehicle i at cell c requests the next cell $c+1$ (or the next cell in an adjacent lane, the diagonal neighbour, if executing a lane change) as soon as it arrives in cell c: $T^{\text{req}}(c+1,i) = T^{\text{arr}}(c,i)$. Requesting on arrival rather than on reaching the cell boundary is what lets a vehicle in free flow cross cell after cell at $\ell/v^{(i)}$ seconds each: the next cell is already held when the vehicle gets there, and the queue in front of an occupied cell is served in request order.
2. **Wait:** If cell $c+1$ is occupied (resource seized by another vehicle), vehicle i waits. The DES scheduler suspends the vehicle's process until the resource becomes available.

3. **Seize:** Upon availability, vehicle i acquires cell $c+1$. The acquisition time is:

$$T^{\text{acq}}(c+1,i) = \max\big(T^{\text{req}}(c+1,i),\ T^{\text{rel}}(c+1,i^-)\big) \tag{3}$$

   where i^- denotes the preceding vehicle and $T^{\text{rel}}(c+1,i^-)$ is the time at which i^- releases cell $c+1$.

4. **Delay:** Vehicle i occupies cell $c+1$ for a travel time of $\ell/v^{(i)}$ seconds.

5. **Release (delayed):** Vehicle i releases the *previous* cell c at time:

$$T^{\text{rel}}(c,i) = T^{\text{acq}}(c+1,i) + \tau^{(i)} \tag{4}$$

   The vehicle holds cell c for an additional $\tau^{(i)}$ seconds (its reaction time) after acquiring cell $c+1$. During this interval, the vehicle simultaneously holds both cells c and $c+1$, representing its physical extent and enforcing a minimum headway.

The arrival time at cell $c+1$ is recorded as:

$$T^{\text{arr}}(c+1,i) = \max\big(T^{\text{acq}}(c+1,i) + \ell/v^{(i)},\ T^{\text{arr}}(c+2,i^-)\big) \tag{5}$$

where $T^{\text{arr}}(c+2,i^-)$ is the time the preceding vehicle arrives in cell $c+2$ and thereby leaves cell $c+1$. The vehicle crosses the cell in $\ell/v^{(i)}$ seconds; if the predecessor is still in the cell at that point, the vehicle waits at the boundary until it has left. In steady-state car following at equal speeds, $T^{\text{acq}}(c+1,i) = T^{\text{acq}}(c+2,i^-) + \tau$ (the delayed release), so the first term of the max, $T^{\text{acq}}(c+2,i^-) + \tau + \ell/v$, exceeds the second, $T^{\text{acq}}(c+2,i^-) + \ell/v$, by exactly τ: the wait is zero at every speed. It binds only when a faster vehicle closes on a slower one in the same lane, where it keeps two vehicles from being in the same cell.

None of these times is calculated explicitly. The simulation never evaluates Eqs. (3)–(5): a vehicle requests, waits, seizes, crosses and releases, and the acquisition, release and arrival times are what that procedure produces.

This protocol repeats for each cell until the vehicle exits the freeway or the simulation ends. The key insight is that no explicit headway-checking rule is needed: the resource contention mechanism inherently prevents a following vehicle from occupying a cell before the leader has released it, which occurs τ seconds after the leader moved on. Congestion, the slowing or stopping of vehicles, arises as a natural consequence of resource contention when request rates exceed release rates.

3.5. Headway Emergence from Resource Protocol

We now show that the delayed-release mechanism produces headways consistent with Newell's car-following model. Consider two consecutive vehicles: leader i^- and follower i, both traveling at speed $v^{(i)}$ in the same lane.

The leader releases cell c at time:

$$T^{\text{rel}}(c,i^-) = T^{\text{acq}}(c+1,i^-) + \tau^{(i^-)} \tag{6}$$

The follower can acquire cell c no earlier than this release time. In steady-state car following, the follower's request time coincides with or follows the release time, giving:

$$T^{\text{acq}}(c,i) = T^{\text{rel}}(c,i^-) = T^{\text{acq}}(c+1,i^-) + \tau^{(i^-)} \tag{7}$$

The time headway at cell c between the two vehicles is:

$$h(c) = T^{\text{arr}}(c, i) - T^{\text{arr}}(c, i^-) = \tau^{(i^-)} + \frac{\ell}{v^{(i)}} \tag{8}$$

This is precisely the headway $h = \tau + d/v$ from Newell's model, with $d = \ell$: the jam spacing is one cell length, since a stopped vehicle occupies exactly one cell (Section 3.1). The headway is not imposed by a rule but *emerges* from the resource acquisition protocol.

3.6. Driver Process

The driver process models the cognitive layer of vehicle behavior, operating concurrently with but decoupled from the movement process. Rather than evaluating at fixed intervals, the driver process uses an event-driven wake-up mechanism: the driver sleeps until either a maximum re-evaluation interval elapses or a neighboring vehicle's movement triggers a wake-up callback. Formally, the driver process waits on:

$$\text{yield} \quad \texttt{timeout}(\delta_{\text{eval}}) \mid \texttt{wake_event} \tag{9}$$

where δ_{eval} is the maximum re-evaluation interval (action interval) and the wake event is triggered externally when a vehicle in the local neighborhood moves. A guard condition prevents redundant wake-ups: if the driver was last evaluated less than $\tau/2$ seconds ago, the wake-up is suppressed.

When a vehicle completes a cell transition (forward or diagonal), it notifies vehicles in its influence zone: the eight neighboring cells (forward, backward, left, right, and diagonals) at both the origin and destination positions. The notification order is physically motivated: for longitudinal moves, upstream neighbors are notified first (they benefit most from the freed cell); for lane changes, the vehicles behind in the target lane are prioritized. The order also gives an implicit prioritisation among vehicles. Drivers woken at the same time instant decide in the order they were notified, and requests made at the same instant for the same cell are queued in the order they are made, so a driver notified earlier is served earlier. This event-driven architecture means that vehicles in free flow with no nearby neighbors generate minimal driver events (only periodic re-evaluations at δ_{eval} intervals), while vehicles in congested regions respond promptly to every local density change.

Upon wake-up, the driver evaluates:

3.6.1. Speed Evaluation

The driver determines a desired speed based on the current traffic context:

- **Free flow:** If no leader is detected within a look-ahead distance (10 cells for HDVs, 12 for AVs), the desired speed is the vehicle's *active maximum speed* on the cell it occupies,

$$v_{\text{des}} = \hat{v}^{(i)}(c) = \min\Big(v_{\max}^{(i)},\ v_{\lim}(c)\Big)\,, \tag{10}$$

  the binding one of two caps: the vehicle's own maximum speed $v_{\max}^{(i)}$ and the speed limit $v_{\lim}(c)$ posted on that cell. The two are distinct: $v_{\max}^{(i)}$ is what the vehicle can do, $v_{\lim}(c)$ is what the road allows there, and $\hat{v}^{(i)}(c)$ is the fastest the vehicle may go there, the speed it would hold with no other vehicle in the way. Every speed the driver chooses on cell c is capped by $\hat{v}^{(i)}(c)$, and the driver chooses again whenever the vehicle

enters a cell with a different limit. The two caps coincide in the experiments of Section 5, where every cell posts 5.2 cells/s and every vehicle has that maximum speed, and they separate wherever a scenario lowers the limit, as a work zone does. For HDVs, a random slowdown is applied with probability p:

$$v^{(i)} = \begin{cases} \max(v_{\text{des}} - \delta v,\, v_{\text{floor}}) & \text{with probability } p \\ v_{\text{des}} & \text{with probability } 1-p \end{cases} \tag{11}$$

where δv is a deceleration increment and $v_{\text{floor}} = 0.5$ cells/s keeps a slowdown from stopping the vehicle. AVs do not experience random slowdown ($p = 0$).

- **Car following:** When a leader is detected, the desired speed is derived from the car-following model. Using Newell's simplified model, the desired spacing behind a leader moving at speed v is:

$$s^*(v) = d^{(i)} + v \cdot \tau^{(i)} \tag{12}$$

Given the current spacing g (in cells, front to front, from the fractional position estimates of the two vehicles) to the leader and the leader's speed v_L, the follower matches the leader's speed and closes the spacing error $g - s^*(v_L)$ within one second, capped by its active maximum speed:

$$v_{\text{target}} = \max\Big(0,\ \min\Big(\hat{v}^{(i)}(c),\ v_L + g - s^*(v_L)\Big)\Big) \tag{13}$$

At the equilibrium $v_{\text{target}} = v_L$ the spacing is exactly Newell's $s^*(v_L)$. When Eq. (13) gives zero behind a leader that is still moving, the follower creeps at 0.1 cells/s so that the spacing can open; a blocked cell ahead is treated as a stopped leader ($v_L = 0$). The target is a closed-form expression, so no iterative look-up is needed and speeds stay continuous, which is a key advantage over integer-speed CA models.

The updated speed takes effect at the next cell transition, modifying the travel time $\ell/v^{(i)}$ for subsequent cells.

3.6.2. Direction Evaluation: Lane Changing

The driver evaluates whether to change lanes based on two mechanisms, each with a logistic probability. Both logistic models, and the conversion of their probabilities to rates given below, are developed in a companion paper [26]; they are restated here as the framework uses them:

Mandatory Lane Change (MLC). A vehicle approaching its destination off-ramp must reach the exit lane (lane 1, the rightmost lane), and a vehicle bound for the end of the segment must reach the lane of its exit there. The probability of initiating a mandatory lane change toward the target lane is modeled as a logistic function of the remaining distance whose midpoint moves with the number of lane changes still required, $n_{\text{LC}} \geq 1$:

$$P_{\text{MLC}}(r, n_{\text{LC}}) = \min\left(1,\ \frac{1 + \exp\big(-k\, n_{\text{LC}}\, r_0\big)}{1 + \exp\big(-k\,(n_{\text{LC}}\, r_0 - r)\big)}\right) \tag{14}$$

where $r = x_{\text{remaining}}/x_{\text{total}}$ is the fraction of the trip remaining, $k > 0$ controls the steepness of the urgency curve, and $r_0 \in (0, 1)$ is the midpoint of the curve for a single lane change. The numerator rescales the logistic curve so that $P_{\text{MLC}} = 1$ at $r = 0$, and below $r = 0.05$ the change is forced ($P_{\text{MLC}} = 1$). As r decreases (vehicle

approaches its exit), the probability increases toward 1. A vehicle that must cross several lanes (e.g., from lane 4 to lane 1, $n_{\mathrm{LC}} = 3$) has its midpoint at $n_{\mathrm{LC}}\, r_0$, three times further from the exit, so its urgency rises earlier. $P_{\mathrm{MLC}} = 0$ for $n_{\mathrm{LC}} = 0$.

Discretionary Lane Change (DLC). A vehicle may change lanes to gain a speed advantage, so the discretionary curve is evaluated only for a neighbouring lane that is faster than the current one, $\Delta v > 0$; a lane with no advantage is never a candidate. The probability is modeled as:

$$P_{\mathrm{DLC}}(\Delta v) = \frac{1}{1 + \exp\big(-k_d\,(\Delta v - \Delta v_0)\big)} \tag{15}$$

where $\Delta v = v_{\text{target lane}} - v^{(i)}$ is the speed advantage in the target lane, the speed of the nearest leader there within the look-ahead distance (the vehicle's own $v_{\max}^{(i)}$ if there is none) minus the vehicle's current speed; k_d controls sensitivity, and Δv_0 is the curve's midpoint, the advantage at which the probability is one half. A lane with a blocked cell ahead is never chosen. A cooldown of 10 s (Table 4) starts whenever the vehicle completes a lane change, mandatory or discretionary. Until it expires the driver does not consider a discretionary change at all, while mandatory changes stay possible. Without it a driver who has just moved into a faster lane could move straight back as soon as the speeds of the two lanes cross, and a few such drivers would keep swapping lanes; the cooldown stops that oscillation without slowing a change the route requires. DLC applies only to HDVs; centrally controlled AVs perform mandatory lane changes only (see Section 3.7.2).

From probability to rate. Drivers do not evaluate at a common frequency in this framework: an AV is evaluated by its controller ten times a second, a human driver about once a second and more often in congestion, where every neighbour's move wakes it. Applying P_{MLC} or P_{DLC} at every evaluation would therefore make the same curve mean different things for different vehicles, since the chance of having acted after n evaluations is $1 - (1 - P)^n$. We read each curve instead as a probability per unit of exposure and convert it to the exposure x accumulated since that driver's previous direction evaluation:

$$q = 1 - (1 - P)^x \tag{16}$$

For the mandatory change the unit is distance, $x = \Delta x / \ell_{\mathrm{ref}}$ with $\ell_{\mathrm{ref}} = v_{\max} \cdot 1\,\mathrm{s} = 5.2$ cells, because an exit is a place and not a time: a vehicle that crawls towards its off-ramp should not attempt more changes per metre than one in free flow. For the discretionary change the unit is time, $x = \Delta t$ in seconds, because the speed advantage it responds to lasts in time; the cooldown stays a refractory period, and exposure is not accumulated through it. The conversion is exact under subdivision, $(1 - P)^{(x/k)\cdot k} = (1 - P)^x$, so the outcome no longer depends on how often a driver happens to be evaluated.

Neither reference is a fitted quantity, and this is what fixes the lane-change parameters in Table 4. Both references are the exposure of a free-flow driver over one second, which is the decision the curves were written for: a human driver re-evaluates about once a second ($\delta_{\mathrm{eval}} = 1.0$ s, Section 5.6), and one second of free flow covers $v_{\max} \cdot 1\,\mathrm{s} = 5.2$ cells, so the distance unit $\ell_{\mathrm{ref}} = 5.2$ cells and the time unit of one second express the same thing, one free-flow driver-second. Read that way, $k = 8.0$, $r_0 = 0.3$, $k_d = 3.0$ and $\Delta v_0 = 1.0$ cells/s state the chance that a driver acts at one such decision: a free-flow HDV evaluating once per second sees $q = P$, while a driver in congestion, woken far more often by its neighbours, accumulates the same exposure over the same distance or

time rather than more attempts. The choice of reference is therefore part of what the four numbers mean and not a free tuning knob: moving to a reference c times longer leaves behaviour unchanged only if every probability is transformed as $1 - P' = (1 - P)^c$.

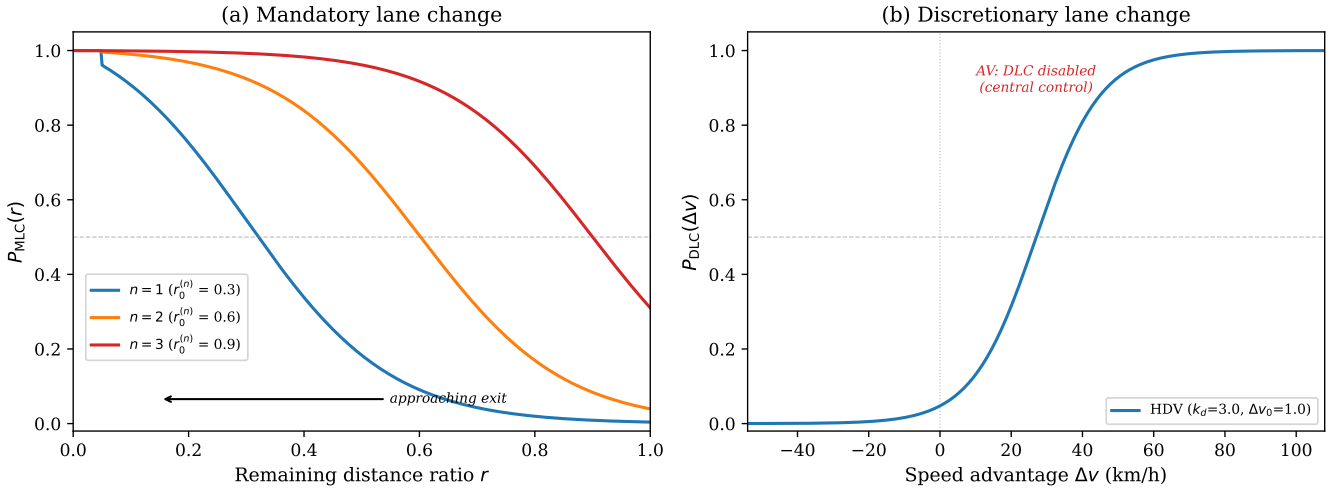


Figure 1: Lane-change logistic probabilities. (a) MLC probability as a function of the remaining distance ratio r for different numbers of required lane changes $n = n_{\mathrm{LC}}$ (Eq. 14). A single lane change ($n = 1$) triggers action close to the exit, while $n = 3$ induces merging much earlier. (b) DLC probability as a function of the speed advantage Δv for HDVs. Centrally controlled AVs do not perform discretionary lane changes.

Figure 1a illustrates how the logistic MLC model captures escalating urgency. A vehicle needing only one lane change can afford to wait until the exit is nearby ($r \approx 0.3$), whereas a vehicle requiring three lane changes begins maneuvering when roughly 90% of its trip remains. Figure 1b shows the HDV discretionary lane-change probability: drivers require a speed advantage of roughly 27 km/h before the lane-change probability reaches 0.5. AVs do not perform discretionary lane changes; under central control, lane changes are executed only when mandatory (blockage avoidance or destination approach), eliminating the merge turbulence that opportunistic lane-seeking creates in mixed traffic.

Lane-Change Safety Assessment. Before executing a lane change, the driver evaluates the safety of the target lane by checking gaps to the nearest leader and follower in that lane. Rather than using fixed safety gaps, the framework employs speed-dependent gaps based on the closing speed between the lane-changing vehicle and its potential neighbors:

$$g_{\mathrm{front}}^{\mathrm{req}} = d + (g_{\mathrm{front}}^{\max} - d) \cdot \frac{\max(0,\ v - v_L)}{v_{\max}^{(i)}} \tag{17}$$

$$g_{\mathrm{rear}}^{\mathrm{req}} = d + (g_{\mathrm{rear}}^{\max} - d) \cdot \frac{\max(0,\ v_F - v)}{v_{\max}^{(i)}} \tag{18}$$

where v is the speed of the lane-changing vehicle i, v_L and v_F are the speeds of the target-lane leader and follower, and $g_{\mathrm{front}}^{\max}$, $g_{\mathrm{rear}}^{\max}$ are the maximum safety gap parameters. The closing speed is normalised by the vehicle's own maximum speed $v_{\max}^{(i)}$, not by its active maximum $\hat{v}^{(i)}(c)$: the ratio measures how fast the two vehicles close relative to the fastest the vehicle can go, so a lowered posted limit does not change how much gap a given closing speed requires. When the closing speed is zero (same speed or diverging), the required gap reduces to the jam spacing d;

when the closing speed is maximal, the full safety gap is required. This formulation avoids both overly conservative gap acceptance (which traps vehicles behind blockages) and overly aggressive merging (which disrupts the target lane).

Blockage-Aware MLC Urgency. When a vehicle detects a blocked cell ahead in its lane, within three times its look-ahead distance, the number of required lane changes n_{LC} in Eq. (14) is inflated by two: $n_{\mathrm{LC}} \leftarrow n_{\mathrm{LC}} + 2$, reflecting that the vehicle must change lane to pass the blockage and then return. For this decision r is the distance to the blocked cell as a fraction of that detection range, and the change is made toward an open neighbouring lane, the right one first. This inflation increases P_{MLC} at greater distances from the blockage, inducing earlier lane changes at higher speeds where natural gaps are available, rather than forcing late merges at low speeds near the blockage.

Lane-Change Execution. When the driver process decides to change lanes, the movement process executes the move at the next cell transition by requesting the diagonal neighbour, the next cell in the target lane, rather than the longitudinal successor in the current lane. One decision produces one lane change: the requested direction returns to forward as soon as the diagonal move is granted, so crossing several lanes takes one decision per lane, each with its own gap check (and, for a discretionary change, its own cooldown). If the target cell is not granted within a patience of 3 s, the request is withdrawn and the vehicle continues forward. During the transition, the vehicle holds resources in both the origin and destination lanes (via delayed release of the origin-lane cell), preventing conflicts.

Vehicles that have been waiting behind a blockage receive elevated priority for cell requests. The mechanism is the DES *PriorityResource* that implements every cell (Section 3.2): each request carries a priority value, and requests waiting for the same cell are served lowest value first, in request order among equal values. Ordinary requests carry 0. A vehicle that has a blocked cell in view requests with $-(1 + t_w)$, where t_w is the time in seconds since it first saw the blockage, so the longer it has waited the earlier it is served. Priority only orders the queue: it never takes a cell from the vehicle that holds it. It ensures that trapped vehicles are not indefinitely starved by through-traffic in adjacent lanes.

3.7. Integrated Framework

The ODCA-DES framework integrates the above components through a dual control architecture that distinguishes between human-driven and autonomous vehicles.

3.7.1. Decentralized Control: Human-Driven Vehicles

Each HDV, upon entering the freeway, spawns two concurrent processes:

1. The **movement process** executes the cell-by-cell request–wait–seize–delay–release cycle (Section 3.4), using the current speed set by the driver process.
2. The **driver process** re-evaluates speed and direction (Section 3.6), updating the vehicle's behavioral state. Re-evaluations are triggered either by the expiration of a maximum action interval δ_{eval} or by wake-up callbacks from neighboring vehicles' movements, whichever occurs first.

These two processes communicate through shared vehicle state: the driver process writes the desired speed and lane-change intention; the movement process reads these values at each cell transition.

3.7.2. Centralized Control: Autonomous Vehicles

AVs do not run individual driver processes. Instead, a **central AV controller**, a single SimPy process, evaluates speed and direction for all active AVs at a fixed update interval, the δ_{eval} of Table 4. At each controller cycle, the controller iterates over all active AVs and executes the same car-following logic used by HDVs, but with AV-specific parameters (τ_{AV}, zero slowdown probability, tighter safety gaps). AVs perform only mandatory lane changes (blockage avoidance, destination approach); discretionary lane changes are disabled. Under central coordination, speed-seeking lane changes are unnecessary, because the controller optimizes longitudinal behavior directly, and their absence eliminates the merge turbulence that arises when individually optimizing vehicles compete for gaps in adjacent lanes.

This architecture reflects the connected nature of autonomous vehicles: rather than each AV independently perceiving and reacting to its local environment, a shared controller with access to all AV states coordinates their behavior. The movement process remains per-vehicle: each AV still independently acquires and releases cell resources through the request–wait–seize–delay–release protocol.

The centralized architecture offers two advantages:

- **Computational efficiency.** A single controller process generating one SimPy event per update cycle replaces N_{AV} independent processes, each generating their own events. This reduces context-switching overhead from $O(N_{\text{AV}})$ to $O(1)$ events per controller cycle.
- **Extensibility.** The controller provides a natural integration point for cooperative AV behaviors (platooning, coordinated merging, and anticipatory speed adjustments) that require global knowledge of the AV fleet state.

3.7.3. Communication Through Shared State

In both control modes, behavioral decisions (speed, direction) are communicated to the movement process through shared vehicle state variables. The decoupling ensures that:

- A vehicle in free flow with no nearby interactions generates minimal events (one seize–release pair per cell, plus periodic driver or controller re-evaluations). Even this can be reduced further, since a vehicle with no one near it need not seize, cross and release every cell one at a time; Section 6.2 lists it as future work.
- A vehicle in congestion generates events driven by resource contention, with speed adjustments in response to changing gaps.
- The movement engine is agnostic to the control mode: it consumes the same state interface regardless of whether decisions originate from a per-vehicle driver process or the central controller.

Figure 2 illustrates the overall architecture.

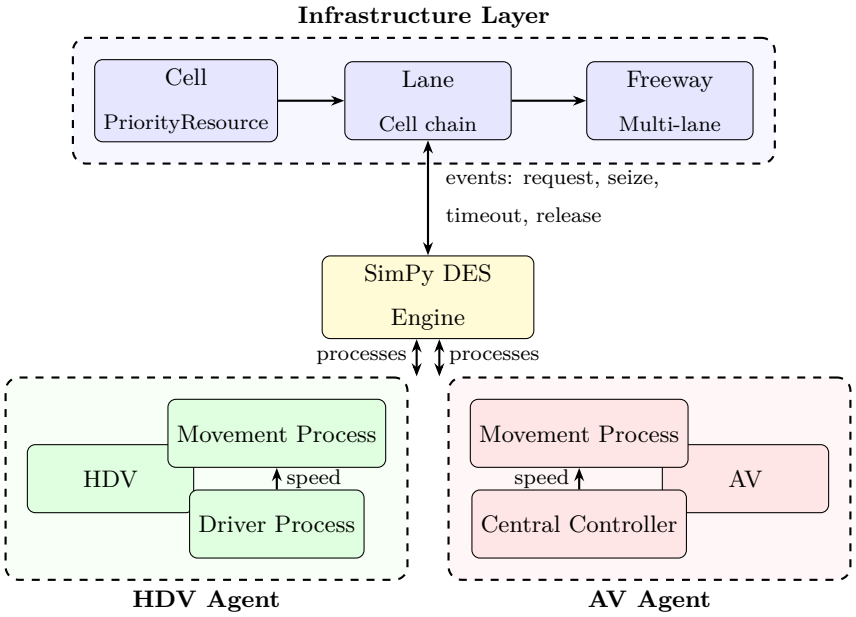


Figure 2: ODCA-DES framework architecture. Infrastructure objects provide the spatial resource structure; the DES engine manages event scheduling and process synchronization. HDVs encapsulate concurrent movement and driver processes, while AVs delegate behavioral decisions to a central controller.

4. Analytical Properties

This section establishes key analytical properties of the ODCA-DES framework, demonstrating its consistency with established traffic flow theory.

4.1. Trajectory Function and Newell's Coordinates

The simulation output is a collection of passage-time records $T^{\text{arr}}(c, i)$ for each vehicle i at each cell c. This is precisely the $T(x, n)$ coordinate system described by Newell [9], where x corresponds to the cell position $c \cdot \ell$ and n corresponds to the vehicle index.

From the passage-time data, all standard traffic flow variables can be derived without spatial or temporal aggregation:

- **Speed** at cell c for vehicle i:
$$v_i(c) = \frac{\ell}{T^{\text{arr}}(c+1, i) - T^{\text{arr}}(c, i)} \tag{19}$$
Note that this is the travel speed (including any waiting time), not the running speed.

- **Headway** at cell c between consecutive vehicles:
$$h(c) = T^{\text{arr}}(c, i) - T^{\text{arr}}(c, i-1) \tag{20}$$

- **Flow** at cell c:
$$q(c) = \frac{1}{h(c)} \tag{21}$$

- **Cumulative count** $N(t, x)$ can be constructed by counting arrivals at position x up to time t, connecting to the LWR theory [30, 31] and its variational formulation [32].

4.2. Fundamental Diagram Derivation

We derive the steady-state fundamental diagram (FD) implied by the ODCA-DES framework, showing that the resource-based movement protocol produces a triangular FD consistent with Newell's kinematic wave model.

4.2.1. Free-Flow Regime

In the absence of resource contention, each vehicle travels at its active maximum speed $\hat{v}^{(i)}(c)$ (Eq. 10). The derivation below treats a homogeneous stream on a uniform road, where every vehicle has the same maximum speed and every cell the same posted limit, so that $\hat{v}^{(i)}(c)$ is one scalar for the whole segment, the free-flow speed v_f; the free-flow speed of traffic flow theory is the special case of the active maximum speed in which vehicles and road are uniform. The travel time per cell is ℓ/v_f, and consecutive cells are acquired without waiting. The density of a single-lane stream of vehicles all traveling at v_f with headway h is:

$$k = \frac{1}{v_f \cdot h} = \frac{1}{v_f \cdot (\tau + \ell/v_f)} = \frac{1}{v_f\tau + \ell} \tag{22}$$

and the flow is:

$$q = k \cdot v_f = \frac{v_f}{v_f\tau + \ell} \tag{23}$$

For low densities where $h > \tau + \ell/v_f$ (i.e., inter-vehicle gaps exceed the minimum headway), vehicles are unconstrained and all travel at v_f. In this regime, flow increases linearly with density:

$$q = k \cdot v_f, \qquad 0 \le k \le k_c \tag{24}$$

where the critical density k_c corresponds to the maximum flow:

$$k_c = \frac{1}{v_f\tau + \ell} \tag{25}$$

4.2.2. Congested Regime

When a vehicle encounters an occupied cell, it waits until the predecessor releases it. In steady-state car following at speed $v < v_f$, the headway is $h = \tau + \ell/v$ (Eq. 8), and the spacing (in cells) is:

$$s = v \cdot h = v\tau + \ell \tag{26}$$

The density is $k = 1/s = 1/(v\tau + \ell)$, giving:

$$v = \frac{1 - k\ell}{k\tau} = \frac{1}{k\tau} - \frac{\ell}{\tau} \tag{27}$$

The flow in the congested regime is:

$$q = k \cdot v = \frac{1}{\tau} - \frac{k\ell}{\tau} = \frac{1}{\tau}(1 - k\ell) \tag{28}$$

This is a linear function of density with slope $-\ell/\tau$, which is precisely the backward wave speed w:

$$w = -\frac{\ell}{\tau} \tag{29}$$

The jam density (when $v = 0$) is:

$$k_j = \frac{1}{\ell} = \frac{1}{d} \tag{30}$$

corresponding to one vehicle per cell with zero speed, exactly the physical constraint imposed by the single-occupancy cell resource. The second equality is the identity $d = \ell$ of Section 3.1: the jam density of Newell's model, one vehicle per jam spacing, is one vehicle per cell.

4.2.3. Capacity and Triangular FD

The capacity (maximum flow) occurs at the critical density k_c, where the free-flow and congested branches meet:

$$q_{\max} = \frac{v_f}{v_f \tau + \ell} \tag{31}$$

The resulting fundamental diagram is triangular (Figure 6):

$$q(k) = \begin{cases} k \cdot v_f & \text{if } k \leq k_c \\ \dfrac{1}{\tau}(1 - k\ell) & \text{if } k_c < k \leq k_j \end{cases} \tag{32}$$

This triangular FD, a two-regime piecewise-linear relationship first motivated by the empirical observations of Greenshields [33], is identical to that derived from Newell's kinematic wave model [9] and the Cell Transmission Model [34]. The key insight is that the two parameters controlling the FD shape, v_f (free-flow speed) and $w = \ell/\tau$ (backward wave speed), emerge directly from the ODCA-DES model parameters: the cell length ℓ and the delayed release time τ. No additional calibration of the FD is needed; it is an inherent consequence of the resource protocol.

4.2.4. Heterogeneous Traffic

When multiple vehicle types share the same lane, each type contributes its own headway parameters (τ, d). In a mixed AV/HDV stream, a follower behind a leader produces headway:

$$h = \tau_{\text{leader}} + \frac{\ell}{v} \tag{33}$$

Note that the headway depends on the *leader's* release delay τ_{leader}, since it is the leader's delayed release that governs when the follower can acquire the cell. In a stream with AV fraction α, the expected headway is:

$$\bar{h}(v) = [\alpha \tau_{\text{AV}} + (1 - \alpha)\tau_{\text{HDV}}] + \frac{\ell}{v} \tag{34}$$

yielding a capacity that increases with AV penetration (since $\tau_{\text{AV}} < \tau_{\text{HDV}}$):

$$q_{\max}(\alpha) = \frac{v_f}{v_f \left[\alpha \tau_{\text{AV}} + (1 - \alpha)\tau_{\text{HDV}}\right] + \ell} \tag{35}$$

This provides a closed-form expression for the capacity benefit of AV penetration under the ODCA-DES framework.

4.3. Comparison with Synchronous Cellular Automata

To clarify the distinctions between the ODCA-DES framework and classical CA models, we present a detailed comparison along three dimensions: speed representation, update synchronization, and conflict resolution. We then illustrate these differences through a worked example.

Figure 3 shows what the two paradigms represent differently, on the same eight vehicles on the same road, both models deterministic so that only the representation differs. The platoon starts at rest at the free-flow spacing, accelerates, meets a posted zone of 21 cells where the limit is 2 cells/s (54 km/h), and accelerates again once it is past; every follower repeats the leader's move a little later and a little further upstream, which is the shape any car-following model must produce. In panel (a) the NaSch platoon exists only at whole seconds and only at whole

cells: the lead vehicle is at cell 64 at $t = 2\,\mathrm{s}$ and at cell 67 at $t = 3\,\mathrm{s}$, and the model says nothing about where it was in between, because there is no in between. In panel (b) the same platoon under ODCA-DES has a mark wherever a vehicle entered a cell, and those instants are whatever the crossing times make them: of the 1,328 cell entries in the window, 14 fall on a whole second. The space axis is identical in both, which is the point of keeping the cell: ODCA-DES relaxes the time axis and the speed value, not the spatial discretisation that makes a CA cheap. Panel (c) follows the last vehicle, the one the zone holds longest. The NaSch speed climbs through the integers 0, 1, 2, 3, 4, 5 and can hold no value between them, so both the approach to the zone and the departure from it are whole-cell steps; the ODCA-DES speed takes 19 distinct values between 1.86 and 5.20 cells/s, slides down through them as the gap to the vehicle ahead closes, holds the posted 2.00 cells/s exactly through all 21 cells of the zone, and returns to the 5.2 cells/s (140 km/h) of Table 4, a speed an integer-cell-per-second model cannot express at all: its nearest representable free-flow speeds are 5 cells/s (135 km/h) and 6 cells/s (162 km/h). The return is a single step rather than a ramp, and the asymmetry is worth naming: the approach to the zone is gradual because the vehicle ahead constrains it through Eq. 1, while nothing constrains a vehicle whose road is clear. Both models are first order, with speed a function of spacing and no vehicle-dynamics layer, so neither bounds acceleration. Bounding it on its own would not refine that: the same spacing rule already prescribes decelerations of up to 1.5 cells/s^2 (11.3 m/s^2) where a free-flowing vehicle meets a standing queue nine cells ahead, so a vehicle held to a plausible acceleration limit but not a braking one would not describe a car. A model with acceleration on both branches means a second-order car-following law in place of Eq. 1, which is outside the scope of this paper.

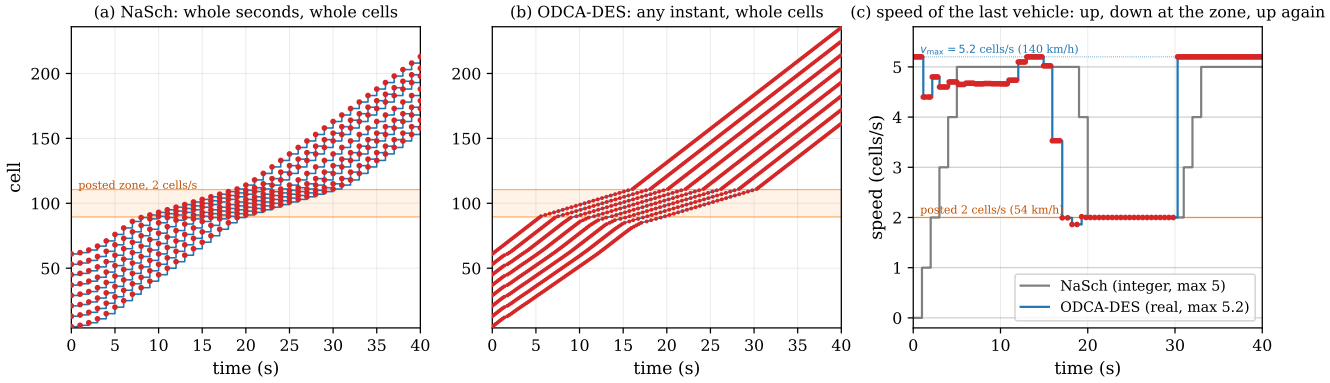


Figure 3: What the two paradigms represent, on the same eight vehicles through the same posted speed-limit zone (shaded, 21 cells at 2 cells/s), both models deterministic (no random slowdown, no driver heterogeneity). The platoon accelerates, slows at the zone and accelerates again, and each follower repeats the move behind the one ahead. (a) NaSch: a red mark is a position, and there is one per vehicle per whole second, each a whole number of cells ahead of the last. (b) ODCA-DES: a red mark is a cell entry, and it occurs at whatever instant the crossing takes; 14 of the 1,328 entries shown fall on a whole second. The cell axis is the same in both. (c) The speed of the last vehicle: NaSch steps through the integers 0 to 5 cells/s, while ODCA-DES passes through 19 distinct values between 1.86 and 5.20, holds the posted 2.00 exactly, and returns to the 5.2 cells/s that the integer model has no way to name.

4.3.1. Speed Representation

In the NaSch model [7], vehicle speed is an integer $v \in \{0, 1, \ldots, v_{\max}\}$ representing cells per timestep. This discretization produces a stepped fundamental diagram: flow can only take values corresponding to integer speed–density combinations, creating artifacts particularly at low densities where vehicles oscillate between adjacent integer

speeds rather than settling at a continuous equilibrium.

In ODCA-DES, speed is continuous-valued, $v^{(i)} \in [0, \hat{v}^{(i)}(c)]$, up to the active maximum speed of Eq. (10). The travel time per cell is ℓ/v, which takes any positive real value. The DES event scheduler handles these continuous inter-event times exactly, producing smooth speed distributions and a continuous fundamental diagram without discretization artifacts.

4.3.2. Update Synchronization

The NaSch model updates all vehicles simultaneously in four sequential sub-steps (acceleration, deceleration, randomization, movement) applied globally at each timestep. This parallel update introduces two artifacts:

1. **Symmetry in interactions:** Two vehicles approaching each other effectively "see" each other's previous-timestep positions, not their current positions. This creates artificial simultaneity in conflict resolution.
2. **Uniform temporal resolution:** All vehicles, regardless of their local traffic context, are updated at the same frequency. A vehicle in free flow with no nearby neighbors receives the same computational attention as a vehicle in a dense platoon.

In ODCA-DES, each vehicle evolves on its own event timeline. A vehicle's next event is scheduled based on its current speed and resource availability, not a global clock. Two vehicles interact only when they compete for the same cell resource, and the interaction is resolved in real time through the DES event calendar. This naturally concentrates computation where interactions occur.

4.3.3. Conflict Resolution

In the NaSch model, conflicts are resolved by explicit rules. The deceleration step checks whether the gap to the leader (measured in cells) is less than the current speed; if so, the speed is reduced to match the gap. This rule-based approach requires careful ordering of sub-steps to avoid inconsistencies.

In ODCA-DES, conflicts are resolved by the resource contention mechanism. A vehicle requesting an occupied cell simply waits: its process is suspended by the DES scheduler until the resource becomes available. No explicit gap-checking is performed. The deceleration effect emerges because the waiting time increases the effective travel time between cells, reducing the measured travel speed. This mechanism is:

- **Deadlock-free:** Vehicles move in one direction, so circular resource dependencies cannot arise.
- **Order-preserving:** The priority resource serves requests of equal priority first come first served, maintaining vehicle ordering within a lane.
- **Physically consistent:** The waiting time exactly corresponds to the time the predecessor physically occupies the cell (via the delayed release), so the resulting headway is consistent with the car-following model.
- **Prioritisable:** Because a request carries a priority value, the same mechanism can serve some vehicles ahead of others waiting for the same cell without any new rule. The model uses it for vehicles that have waited long behind a blockage (Section 3.6.2); special vehicles such as ambulances could be given priority the same way.

4.3.4. Illustrative Example

Consider three vehicles (A, B, C) on a single-lane road with cells indexed $1, 2, \ldots, 10$; a vehicle leaves the road on arriving at cell 10 and releases that cell τ later. At time $t = 0$: vehicle A is at cell 8 with speed $v_A = 2$ cells/s, vehicle B is at cell 5 with speed $v_B = 3$ cells/s, and vehicle C is at cell 1 with speed $v_C = 3$ cells/s. Parameters: $\tau = 1$ s, $\ell = 1$ cell. Both evolutions below, and Figure 4, are computed from the rules as stated.

NaSch evolution ($\Delta t = 1$ s, $v_{\max} = 3$). At each timestep, all vehicles simultaneously execute: (1) accelerate by 1 (up to $v_{\max}$), (2) decelerate to the number of empty cells ahead if that gap is smaller than the speed, (3) randomize, (4) move. Random slowdown is switched off ($p = 0$) so that the example is deterministic. Table 1 traces the resulting state evolution.

Table 1: NaSch synchronous update for three vehicles. Speeds are integers; all updates occur simultaneously at each timestep. *Deceleration applied due to gap constraint.

t	Vehicle A		Vehicle B		Vehicle C	
	cell	v	cell	v	cell	v
0	8	2	5	3	1	3
1	exit	–	7	2*	4	3
2	–	–	10	3	6	2*
3	–	–	exit	–	9	3

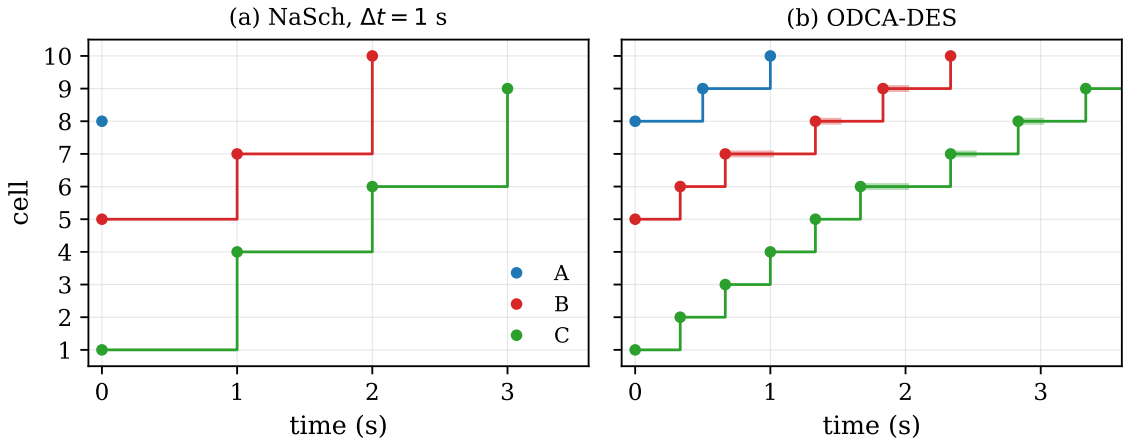


Figure 4: The three-vehicle example in space and time. (a) NaSch: positions exist only at whole seconds and jump by whole cells (Table 1). (b) ODCA-DES: a mark is the arrival in a cell (Table 2); a thick pale segment is a wait between requesting the next cell and seizing it, while the vehicle ahead still holds it.

At $t = 1$, vehicle B slows to 2 cells/s and stops at cell 7 because it sees A at cell 8, A's position at $t = 0$, although A leaves the road in the same update: under the synchronous update every vehicle reacts to the state of the previous step. At $t = 2$, C is held back in the same way by B's position at $t = 1$. Speed adjustments are abrupt (integer steps), and the gap-checking rule must be carefully ordered to avoid collisions.

ODCA-DES evolution. Each vehicle follows the protocol of Section 3.4 on its own timeline. At $t = 0$ each requests its next cell and, the cell being free, seizes it at once: A seizes cell 9, B cell 6 and C cell 2. Each then crosses into it in ℓ/v seconds, 0.5 s for A and 0.333 s for B and C. Events are processed in chronological order, as shown in Table 2.

Table 2: ODCA-DES event sequence for three vehicles up to $t = 1.5$ s. Times are continuous; events are processed in chronological order. A vehicle requests its next cell as soon as it arrives in a cell, and advances independently until resource contention occurs.

Time (s)	Event	Vehicle	Notes
0.000	Acquire cells 9, 6, 2	A, B, C	A@8, B@5, C@1; all free
0.333	Arrive cell 6, acquire 7	B	Free
0.333	Arrive cell 2, acquire 3	C	Free
0.500	Arrive cell 9, acquire 10	A	Free
0.667	Arrive cell 7, request 8	B	**Wait**: A holds cell 8 until $t = 1.0$
0.667	Arrive cell 3, acquire 4	C	Free
1.000	Release cell 8	A	Delayed release: $T^{\text{acq}}(9, A) + \tau = 0 + 1.0$
1.000	Acquire cell 8	B	Resource freed; B seizes
1.000	Release cell 5	B	$T^{\text{acq}}(6, B) + \tau = 0 + 1.0$
1.000	Arrive cell 4, acquire 5	C	Released by B at $t = 1.0$
1.000	Arrive cell 10, leave road	A	Holds cell 10 until $t = 2.0$
1.333	Arrive cell 8, request 9	B	**Wait**: A holds cell 9 until $t = 1.5$
1.333	Arrive cell 5, acquire 6	C	Released by B at $t = 1.333$
1.500	Release cell 9	A	$T^{\text{acq}}(10, A) + \tau = 0.5 + 1.0$
1.500	Acquire cell 9	B	Resource freed; B seizes

Several key differences are visible:

1. **Continuous timing:** Events occur at $t = 0.333, 0.500, 0.667, \ldots$ rather than at integer timesteps, reflecting the continuous speeds.
2. **Asynchronous interaction:** Each vehicle generates events on its own timeline without waiting for a global clock tick, and interacts only with the vehicle directly ahead, through the cells that vehicle releases: C never waits for A, and waits for B only from cell 7 on (Figure 4b).
3. **Resource-mediated conflict:** Vehicle B does not "check its gap" to A. Instead, it requests cell 8, finds it occupied, and waits. The delay emerges from A's release schedule, not from an explicit deceleration rule.
4. **Headway emergence:** B acquires cell 8 at $t = 1.0$, exactly $\tau = 1.0$ s after A acquired cell 9 at $t = 0$, and arrives in it at $t = 1.333$, which is $\tau + \ell/v_B$ after A was there at $t = 0$. The headway $h = \tau + \ell/v$ of Eq. (8) is produced by the resource protocol without any headway-enforcement rule.

Table 3 summarizes the structural differences between the two frameworks.

Table 3: Structural comparison of the NaSch CA model and the ODCA-DES framework.

Dimension	NaSch CA	ODCA-DES
Speed domain	Integer $\{0, 1, \ldots, v_{\max}\}$	Continuous $[0, \hat{v}^{(i)}(c)]$
Time advancement	Fixed Δt (synchronous)	Event-to-event (asynchronous)
Update order	Parallel (all vehicles)	Per-vehicle (own timeline)
Conflict resolution	Rule-based gap check	Resource contention (wait)
Headway control	Explicit deceleration rule	Emerges from delayed release
FD shape	Stepped (integer artifacts)	Smooth (triangular)
Coordinate system	$X(t, n)$ via position update	$T(x, n)$ via passage times
Heterogeneity	Difficult (same update cycle)	Natural (per-vehicle parameters)

5. Computational Experiments

This section presents computational experiments designed to validate the analytical properties derived in Section 4 and demonstrate the framework's capabilities for modeling heterogeneous multi-lane freeway traffic. All experiments are implemented using the ODCA-DES framework built on SimPy [20] in Python 3.13. Random number generation uses NumPy's `SeedSequence` to produce independent per-source streams.

5.1. Experimental Setup

5.1.1. Network Configuration

The test network is a four-lane unidirectional freeway segment of length $L = 6$ km (800 cells at $\ell = 7.5$ m per cell). Lanes are indexed from 1 (rightmost) to 4 (leftmost). The network includes:

- **On-ramp 1:** at cell 100 (750 m from the start), merging into lane 1.
- **On-ramp 2:** at cell 400 (3.0 km from the start), merging into lane 1.
- **Off-ramp 1:** at cell 300 (2.25 km), exiting from lane 1.
- **Off-ramp 2:** at cell 550 (4.125 km), exiting from lane 1.
- **Off-ramp 3:** at cell 750 (5.625 km), exiting from lane 1.

The mainline entry (cell 1) is the primary source, with vehicles distributed across all four lanes according to an initial lane-assignment probability proportional to lane capacity. Figure 5 illustrates the network geometry.

5.1.2. Vehicle Parameters

Two vehicle types are modeled: human-driven vehicles (HDVs) and autonomous vehicles (AVs). Table 4 summarizes the behavioral parameters for each type.

The primary distinction is the reaction time: $\tau_{\text{HDV}} = 1.5$ s vs. $\tau_{\text{AV}} = 0.5$ s. This directly controls headway (Eq. 8) and, consequently, lane capacity (Eq. 31). HDVs additionally experience random slowdowns ($p = 0.15$), introducing stochastic speed fluctuations absent in AVs.

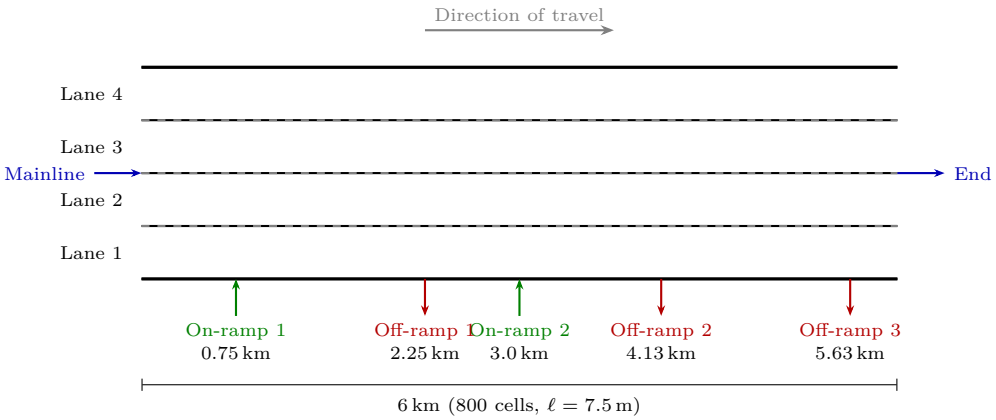


Figure 5: Test network geometry: 4-lane unidirectional freeway segment (6 km, 800 cells at $\ell = 7.5$ m). On-ramps merge into lane 1 (rightmost); off-ramps exit from lane 1. Mainline demand enters at the first cell of each lane; traffic bound for the segment end is spread over the four lane ends (cell 800), so it crosses lanes in both directions.

To capture the heterogeneity of real driver populations, each HDV is assigned individual behavioral parameters at creation time by sampling from distributions centered on the mean values in Table 4:

- Reaction time: $\tau_i \sim \text{LogNormal}(\mu_\tau, \sigma_\tau)$, clipped to $[0.5, 3.0]$ s.
- Action interval: $\delta_i \sim \text{LogNormal}(\mu_\delta, \sigma_\delta)$, clipped to $[0.3, 3.0]$ s.
- Slowdown probability: $p_i \sim \mathcal{N}(\bar{p}, \sigma_p)$, clipped to $[0, 1]$.

The LogNormal distribution is chosen for τ and δ because both quantities are strictly positive and right-skewed in empirical observations. AVs use deterministic parameters (zero standard deviation), reflecting the consistency of automated control systems. This per-driver heterogeneity introduces capacity scatter in the fundamental diagram (Section 5.2) and more realistic platoon dynamics compared to homogeneous-driver models.

5.1.3. Demand and Scenarios

The simulation duration is $T_{\text{sim}} = 3600$ s (1 hour). Origins and destinations are named places on the network: four mainline entries (one per lane, cell 0), two on-ramps (lane 1 at cells 100 and 400), three off-ramps (lane 1 at cells 300, 550 and 750) and one exit per lane at the segment end (cell 800). Demand is given as a flow in veh/h for each origin–destination pair, and one generator per pair releases vehicles into its origin cell; a pair whose origin cell is occupied queues, so the road itself limits what enters.

Mainline demand is 6,000 veh/h (1,500 per lane) and each on-ramp contributes 500 veh/h. The split over destinations reflects lane-use patterns: the rightmost lanes carry most of the off-ramp traffic, the leftmost lanes mostly through traffic. Lane 1 sends 20% of its vehicles to off-ramp 1, 25% to off-ramp 2 and 15% to off-ramp 3, keeping 40% through; lane 4 sends 2%, 5% and 3% to the three off-ramps and keeps 90% through. On-ramp 1 splits between off-ramp 2 (50%), off-ramp 3 (20%) and the segment end (30%); on-ramp 2 between off-ramp 3 (30%) and the segment end (70%). Through traffic is spread evenly over the four lane ends, so a share of every origin has to cross lanes in each direction rather than drive straight to the end of its entry lane. A vehicle bound for an off-ramp

Table 4: Vehicle behavioral parameters used in the computational experiments.

Parameter	Symbol	HDV	AV
Reaction time / release delay	τ	1.5 s	0.5 s
Jam spacing ($= \ell$)	d	1 cell (7.5 m)	1 cell (7.5 m)
Maximum speed	$v_{\max}$	5.2 cells/s (140 km/h)	5.2 cells/s (140 km/h)
Random slowdown probability	p	0.15	0.0
Slowdown decrement	δv	0.5 cells/s	–
MLC steepness	k	8.0	8.0
MLC midpoint	r_0	0.3	0.3
DLC sensitivity	k_d	3.0	– (disabled)
DLC midpoint	Δv_0	1.0 cells/s	– (disabled)
DLC cooldown	–	10.0 s	– (disabled)
MLC exposure reference	ℓ_{ref}	5.2 cells (1 s at $v_{\max}$)	5.2 cells (1 s at $v_{\max}$)
DLC exposure reference	–	1.0 s	– (disabled)
Action interval (max. re-evaluation gap)	δ_{eval}	1.0 s	0.1 s (controller cycle)
Front safety gap	$g_{\text{front}}^{\max}$	2.0 cells	1.0 cell
Rear safety gap	$g_{\text{rear}}^{\max}$	2.0 cells	1.0 cell
Per-driver heterogeneity (HDV only)			
τ std. dev.	σ_τ	0.3 s	0 (deterministic)
Action interval std. dev.	σ_δ	0.2 s	0 (deterministic)
Slowdown prob. std. dev.	σ_p	0.05	0 (deterministic)

must be in lane 1 when it gets there; a vehicle bound for the end of lane j must be in lane j. A vehicle that reaches the last cell in the wrong lane still leaves the network there, and the run records it as a missed exit.

An endpoint's road cell is a cell like any other, which is what lets a scenario throttle it: a speed limit on that cell caps the flow through it at $3600/(\tau + \ell/v)$ veh/h, an artificial bottleneck at a ramp or at the end of the segment. In the experiments reported here every endpoint is transparent (no speed limit, no time spent), so the network is the one described above. Capacity reductions inside the segment are modelled instead as *incidents*: a set of cells is closed or slowed for a stated interval and then restored, which is how the lane-drop of Section 5.8 is created.

Four scenarios, summarized in Table 5, are evaluated to isolate the effects of AV penetration:

Each scenario is run with 20 independent replications (seeds 1–20) to account for stochastic variability in vehicle generation, type assignment, and HDV random slowdowns. All reported metrics are sample means with 95% confidence intervals.

5.1.4. Implementation and Hardware

The framework is implemented in Python (3.13) on SimPy (4.1), with one process per vehicle, one per human driver and one for the AV controller; random numbers come from one NumPy `SeedSequence` stream per source of

Table 5: Experimental scenarios varying AV penetration rate.

Scenario	Description	AV Penetration (α)
S1	Baseline (all HDV)	0%
S2	Low AV penetration	30%
S3	Moderate AV penetration	50%
S4	High AV penetration	70%

randomness, so a run is reproducible from its seed alone. Every run is single-threaded; the replications reported below were executed in parallel.

5.2. Fundamental Diagram Validation

5.2.1. Single-Lane Homogeneous Traffic

To validate the theoretical FD derived in Section 4.2, we simulate a single-lane closed segment (400 cells, no ramps) initialized at various densities ranging from $k = 0.01$ to 0.80 veh/cell. Vehicles are placed uniformly across the segment with an inflow process that replaces exiting vehicles to maintain the target density. Each configuration runs for 1200 s with a 200 s warm-up period. Flow–density pairs are computed using Edie's generalized definitions [35] over a measurement region (cells 100–300) with 20-second aggregation windows:

$$q = \frac{D}{A}, \qquad k = \frac{W}{A} \tag{36}$$

where D is the total distance traveled by all vehicles within the space-time region of area $A = \Delta x \cdot \Delta T$, and W is the total time spent by all vehicles within the region. This density-initialization approach produces a complete fundamental diagram spanning free-flow through jam density, and Edie's definitions ensure consistency between flow, density, and speed measurements.

The theoretical capacity for these parameters is:

$$q_{\max} = \frac{5.2}{5.2 \times 1.5 + 1.0} = \frac{5.2}{8.8} \approx 0.591 \text{ veh/cell/s} = 2127 \text{ veh/h} \tag{37}$$

and the backward wave speed is $w = \ell/\tau = 1.0/1.5 = 0.667$ cells/s = 18 km/h, consistent with empirically observed congestion wave speeds [36].

Two measurements are worth separating here. On a ring road, where no vehicle enters or leaves, the four configurations tested on the ring-road control (deterministic, slowdowns only, heterogeneity only, full stochastic) peak at 1,948 to 2,041 veh/h, that is 92 to 96% of the analytic $q_{\max}$; these are the largest single measurement intervals in each scatter, and the configurations differ by less than the scatter of the diagram itself. The shortfall from the closed form is the discreteness it ignores, since a vehicle occupies whole cells and releases each one τ seconds after the next is acquired. On the open segment measured below, flow tops out at 1,586 veh/h instead, because vehicles have to enter through a single first cell and the measurement region sees the start-up wave that follows; the free-flow branch still lies on the theoretical line.

5.2.2. Comparison with Integer-Speed CA

To demonstrate the advantage of continuous speeds, we implement a NaSch-equivalent simulation on the same single-lane segment with integer speeds $v \in \{0, 1, 2, 3, 4\}$ and synchronous updates at $\Delta t = 1\,\text{s}$. Figure 6 overlays the resulting flow–density points for both models against the theoretical triangular FD. ODCA-DES (top row) yields a smooth envelope: the free-flow branch lies on the theoretical line, the apex is reached at 1,586 veh/h (the open-boundary value discussed above), and the congested branch descends toward jam density along the theoretical slope, without the artefacts of integer speeds. The NaSch model (bottom row) produces a staircase pattern in the congested branch due to integer-speed quantization, with flow–density clusters corresponding to each integer speed level. The zoomed panels (b, d) highlight the near-capacity region where the difference is most pronounced.

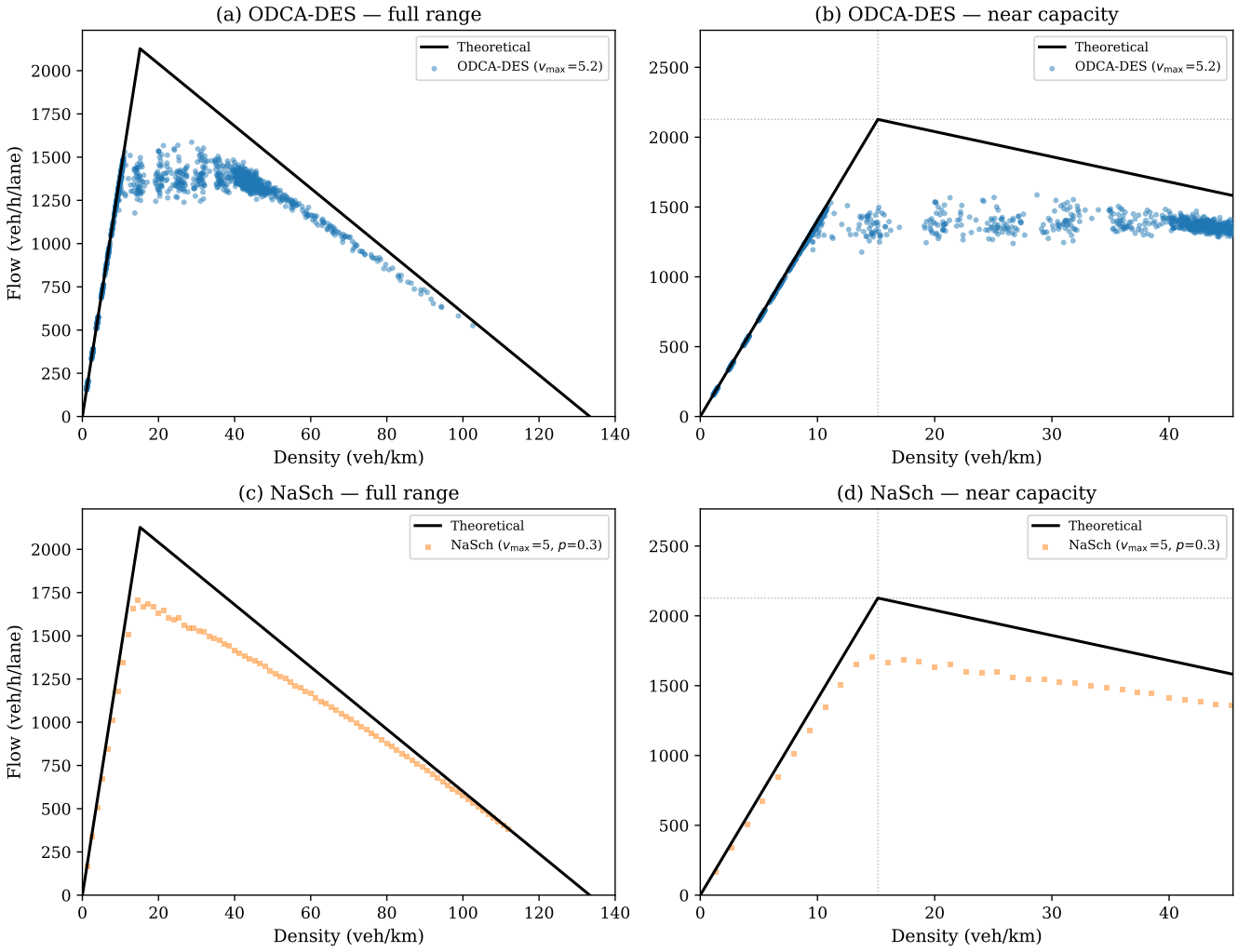


Figure 6: Fundamental diagram validation against the theoretical triangular FD ($v_f = 5.2\,\text{cells/s}$, $\tau = 1.5\,\text{s}$, $q_{\max} = 2127\,\text{veh/h}$). Top row: ODCA-DES (continuous speeds, $v_{\max} = 5.2$) produces a smooth envelope across (a) the full density range and (b) near capacity, on the theoretical line in free flow and peaking at 1,586 veh/h. The peak is low because this is an open segment: the single entry cell and the start-up wave behind it hold the flow to 1,586 veh/h, while on a ring road, where no vehicle enters or leaves, the peak is 1,948 to 2,041 veh/h, 92 to 96% of $q_{\max}$. Bottom row: NaSch (integer speeds, $v_{\max} = 5$, $p = 0.3$) produces clustered flow–density points with a staircase congested branch across (c) the full range and (d) near capacity.

5.3. Trajectory Analysis

The $T(x, n)$ passage-time output enables direct construction of time-space diagrams. For each vehicle, the trajectory is the set of points $\{(T^{\text{arr}}(c, i),\ c \cdot \ell)\}$, plotted with time on the horizontal axis and position on the vertical axis.

5.3.1. Free-Flow Trajectories

Under low demand (the baseline parameters of Table 4 with mainline demand reduced to 3,000 veh/h), vehicles traverse the segment without significant resource contention. Figure 7a shows the resulting time-space diagram: trajectories are near-parallel lines with slope ℓ/v_f, confirming that the event-driven movement process produces uniform free-flow speeds with only minor perturbations from HDV random slowdowns.

5.3.2. Congestion Formation and Wave Propagation

Under high demand (S1, 6000 veh/h mainline + ramp flows), congestion forms upstream of off-ramps where lane-changing vehicles create friction. As shown in Figure 7b, the time-space diagram reveals disturbances that move upstream: each vehicle slows where the one ahead slowed, a short distance later in space and time, so the pattern of slope changes drifts backwards against the traffic. Nothing in the model schedules this; it follows from a cell being released τ after its holder has moved on.

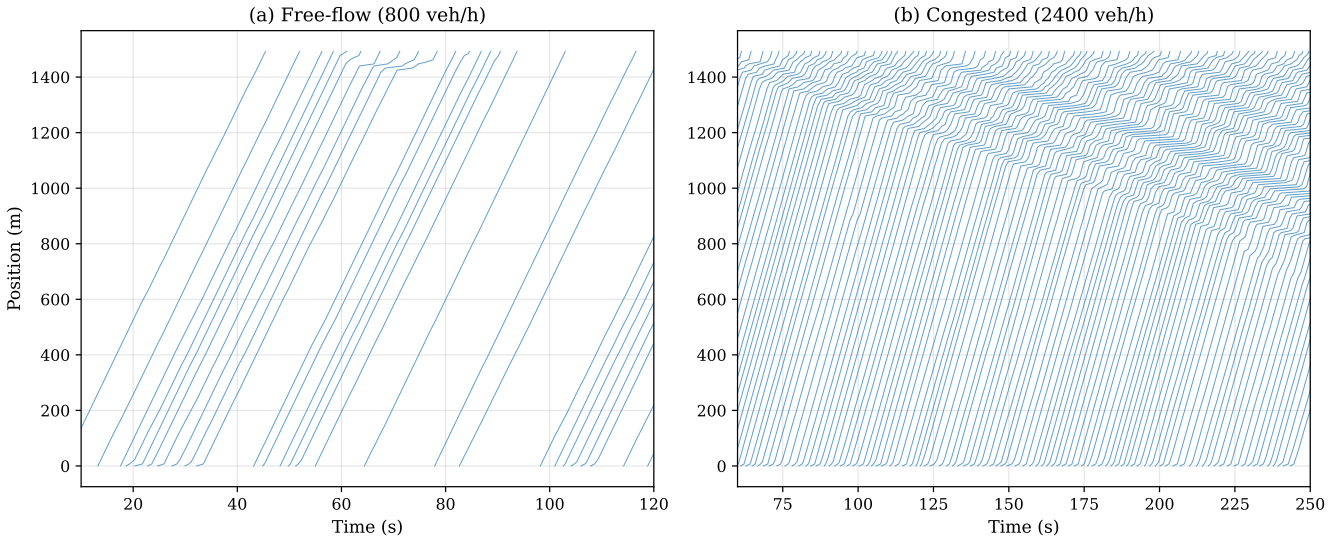


Figure 7: Time-space diagrams on a single-lane segment. (a) Free-flow (800 veh/h): trajectories are near-parallel with slope $v_f = 5.2$ cells/s (≈140 km/h), with minor perturbations from HDV random slowdowns. (b) Congested (2400 veh/h): trajectory slopes decrease as density exceeds capacity, with backward-propagating waves visible as slope transitions. Trajectories are coloured by each vehicle's average speed (blue above 100 km/h, orange 40–100, red below 40); at this demand every vehicle averages above 100 km/h, and the congestion shows as the bending of individual trajectories rather than as slow vehicles. The disturbances visible in panel (b) travel upstream, the direction the theory gives for $w = \ell/\tau = 0.667$ cells/s (≈18 km/h).

5.3.3. Travel Speed vs. Running Speed

The passage-time data naturally distinguishes between travel speed (including waiting) and running speed (movement only). For vehicle i at cell c:

$$v_{\text{travel}}(c,i) = \frac{\ell}{T^{\text{arr}}(c+1,i) - T^{\text{arr}}(c,i)} \tag{38}$$

$$v_{\text{run}}(c,i) = \frac{\ell}{T^{\text{arr}}(c+1,i) - T^{\text{acq}}(c+1,i)} \tag{39}$$

The difference $v_{\text{run}} - v_{\text{travel}}$ quantifies the delay experienced at each cell. Aggregating this difference across cells and vehicles provides a spatial delay profile, identifying bottleneck locations without requiring predefined detector stations. Figure 8 plots both metrics as a function of position along the freeway. Both fall in the first four kilometres, where the on-ramps feed in and the off-ramp traffic works its way to lane 1, and recover to near free flow after off-ramp 2 at 4.125 km, by which point much of the demand has left the segment. The gap between the two curves is a few km/h: at this demand most of the lost time is spent travelling slowly behind a leader rather than waiting for a cell to be released.

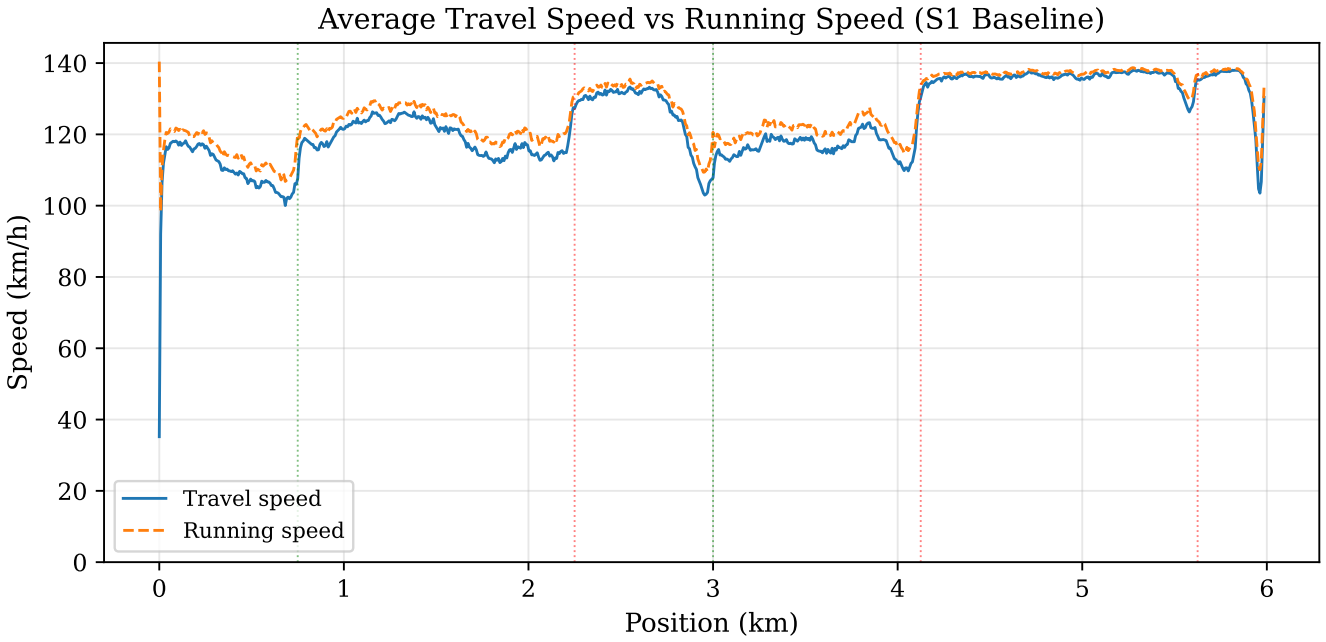


Figure 8: Average travel speed (solid) and running speed (dashed) as a function of position along the freeway (S1, one run). The gap between the two curves is the average cell-level waiting time. Speeds are lowest over the first four kilometres, where ramp traffic and mandatory lane changes concentrate, and recover downstream of off-ramp 2. Dotted lines mark the on-ramps, dashed lines the off-ramps.

5.4. Mixed Traffic Scenarios

Scenarios S1–S4 are compared across nine performance metrics, measured over the full simulation period after a 300-second warm-up interval. All values are means over 20 independent replications (seeds 1–20); 95% confidence intervals are reported.

5.4.1. Performance Metrics

1. **Throughput** (Q): vehicles leaving the network (via off-ramps and segment end) during the measurement period, counted by their exit time and scaled to veh/h.
2. **Average travel time** ($\overline{TT}$): mean time from entry to exit, over the same vehicles.
3. **Average delay** ($\overline{D}$): mean excess over free-flow travel, summed cell by cell as

$$D_i = \sum_c \max\left(0,\ T^{\text{arr}}(c+1,i) - T^{\text{arr}}(c,i) - \frac{\ell}{\hat{v}^{(i)}(c)}\right), \tag{40}$$

   where $\hat{v}^{(i)}(c)$ is the active maximum speed of Eq. 10, the vehicle's own maximum speed capped by the limit posted on the cell it is crossing. Delay is therefore what other vehicles cost this one: a cell crossed at its posted limit adds nothing, and a lowered limit changes the free-flow travel time rather than the delay. Counting per cell rather than as total time minus distance over v_f keeps the extra distance driven by a vehicle that missed its off-ramp out of its delay.
4. **Lane-change frequency** ($\overline{LC}$): lane changes per vehicle per km, over the distance the vehicle actually drove.
5. **Vehicles delayed** $> 20\,\text{s}$ (%): fraction of vehicles experiencing total delay exceeding 20 s.
6. **Mean cells held**: the mean number of cells a vehicle holds while it drives, the one it occupies plus those its delayed releases still lock behind it, averaged over its travel time.
7. **Mean origin wait** (s): the mean time a vehicle waited at its origin before it got its first cell.
8. **Never entered** (veh): vehicles generated during the run that never got on the road.
9. **Gap rejections**: lane-change attempts the driver's gap check refused, a target cell already held included, counted over the whole run. The other lane-change failure the run counts, a vehicle giving up a change after waiting out its patience, is zero in every run of every scenario and is not tabulated.

The first seven are measured over the same set of vehicles: those whose exit time falls in the measurement window; never entered counts every vehicle the run generated, and gap rejections is a counter over the run. A vehicle already on the road when the warm-up ends therefore counts, which matters at high demand where a large share of the vehicles in the network at any moment entered before the window.

5.4.2. Results

Table 6 reports the performance metrics across the four scenarios.

Based on the analytical capacity expression (Eq. 35), the expected theoretical lane capacities are:

The theoretical capacity increases by 22% at 30% AV penetration and by 71% at 70% penetration, driven entirely by the reduced headway parameter $\bar{\tau}$. Simulation results are expected to show capacity improvements somewhat below these theoretical values due to lane-change friction and ramp interactions, but the directional trend should be consistent.

5.4.3. Discussion

The simulation results confirm the expected trends from the analytical capacity predictions:

Table 6: Performance metrics across AV penetration scenarios ($T_{\text{sim}} = 3600\,\text{s}$, $N = 20$ replications). Values are means ± 95% confidence intervals. The network operates near capacity with per-lane OD flows and three off-ramps requiring mandatory lane changes, creating congested baseline conditions.

Metric	S1 (0%)	S2 (30%)	S3 (50%)	S4 (70%)
Throughput (veh/h)	$5{,}885 \pm 24$	$6{,}689 \pm 25$	$6{,}971 \pm 33$	$6{,}983 \pm 35$
Avg. travel time (s)	345.5 ± 4.8	243.2 ± 10.9	156.0 ± 1.0	140.1 ± 0.3
Avg. delay (s)	214.6 ± 4.8	111.9 ± 11.0	24.6 ± 1.0	8.6 ± 0.3
LC frequency (/veh/km)	2.32 ± 0.01	1.79 ± 0.04	0.99 ± 0.01	0.55 ± 0.01
Vehicles delayed $> 20\,\text{s}$ (%)	99.8 ± 0.1	96.9 ± 0.7	57.9 ± 2.9	5.9 ± 0.7
Mean cells held	4.18 ± 0.04	4.54 ± 0.14	5.39 ± 0.03	4.90 ± 0.01
Mean origin wait (s)	89.7 ± 8.3	2.9 ± 0.2	1.3 ± 0.1	0.6 ± 0.0
Never entered (veh)	659 ± 44	9.2 ± 7.4	2.1 ± 1.2	0.8 ± 0.6
Gap rejections ($\times 10^3$)	642.3 ± 3.1	419.3 ± 11.3	224.9 ± 3.3	130.5 ± 1.9

Table 7: Theoretical lane capacity as a function of AV penetration rate, computed from Eq. (35) with $v_f = 5.2\,\text{cells/s}$, $\tau_{\text{HDV}} = 1.5\,\text{s}$, $\tau_{\text{AV}} = 0.5\,\text{s}$, $\ell = 1\,\text{cell}$.

α	$\bar{\tau}$ (s)	k_c (veh/cell)	$q_{\max}$ (veh/cell/s)	$q_{\max}$ (veh/h/lane)
0%	1.50	0.114	0.591	2127
30%	1.20	0.138	0.718	2586
50%	1.00	0.161	0.839	3019
70%	0.80	0.194	1.008	3628

Throughput and the demand ceiling. Network throughput increases with AV penetration, from $5{,}885 \pm 24\,\text{veh/h}$ at $\alpha = 0\%$ to $6{,}983 \pm 35\,\text{veh/h}$ at $\alpha = 70\%$, an 18.7% improvement, and the confidence intervals are tight (coefficient of variation 0.8–1.1%), so 20 replications resolve the differences comfortably. Most of the gain is already realised at 30% AV ($6{,}689 \pm 25$, that is 13.7%), and the last two scenarios are 0.2% apart. The reason is that the network runs out of demand before it runs out of capacity: mainline and ramps offer 7,000 veh/h, so 6,983 veh/h at $\alpha = 70\%$ is 99.8% of everything that asks to enter. The all-HDV baseline, by contrast, serves 84.1% of the offered demand and holds the rest back in the origin queues. The last three rows of Table 6 show that queue directly: an S1 vehicle waits $89.7 \pm 8.3\,\text{s}$ at its origin and 659 ± 44 vehicles per run never get on, against 0.6 s and one vehicle at 70% AV. Mean cells held rises from 4.2 in S1 to 5.4 in S3 and falls back to 4.9 in S4: the faster the stream, the more cells a vehicle's delayed releases lock behind it, until the AV's shorter τ takes them back. The measured improvement is therefore a lower bound on the capacity effect: the theoretical 71% capacity increase (Table 7) cannot show up in a throughput figure that the demand caps. Where the extra capacity does show up is in delay, which keeps falling after throughput has saturated. Figure 9 visualizes the throughput trend with 95% confidence intervals.

Delay. Average delay drops from 214.6 ± 4.8 s to 8.6 ± 0.3 s as AV penetration increases from 0% to 70%, a 96% reduction, and the share of vehicles delayed by more than 20 s falls from essentially all of them (99.8%) to 5.9%. The high baseline delay reflects the congested operating conditions: the per-lane OD flows with destination distributions require substantial lane-changing activity, and the combined mainline and ramp demand (7,000 veh/h) loads the network near its effective capacity. At 70% AV penetration, the shorter headway ($\tau = 0.5$ s vs. 1.5 s) is what drives the car-following delay down. Figure 10 presents the average travel time across scenarios.

Lane changes. Lane-change frequency *decreases* with AV penetration (Table 6). Because AVs perform only mandatory lane changes, increasing AV penetration eliminates discretionary lane-change friction. The remaining lane changes are driven entirely by route requirements (off-ramp exits), and AVs' shorter headways ($\tau = 0.5$ s) create more available gaps for these mandatory maneuvers, reducing queuing behind slower vehicles in the target lane.

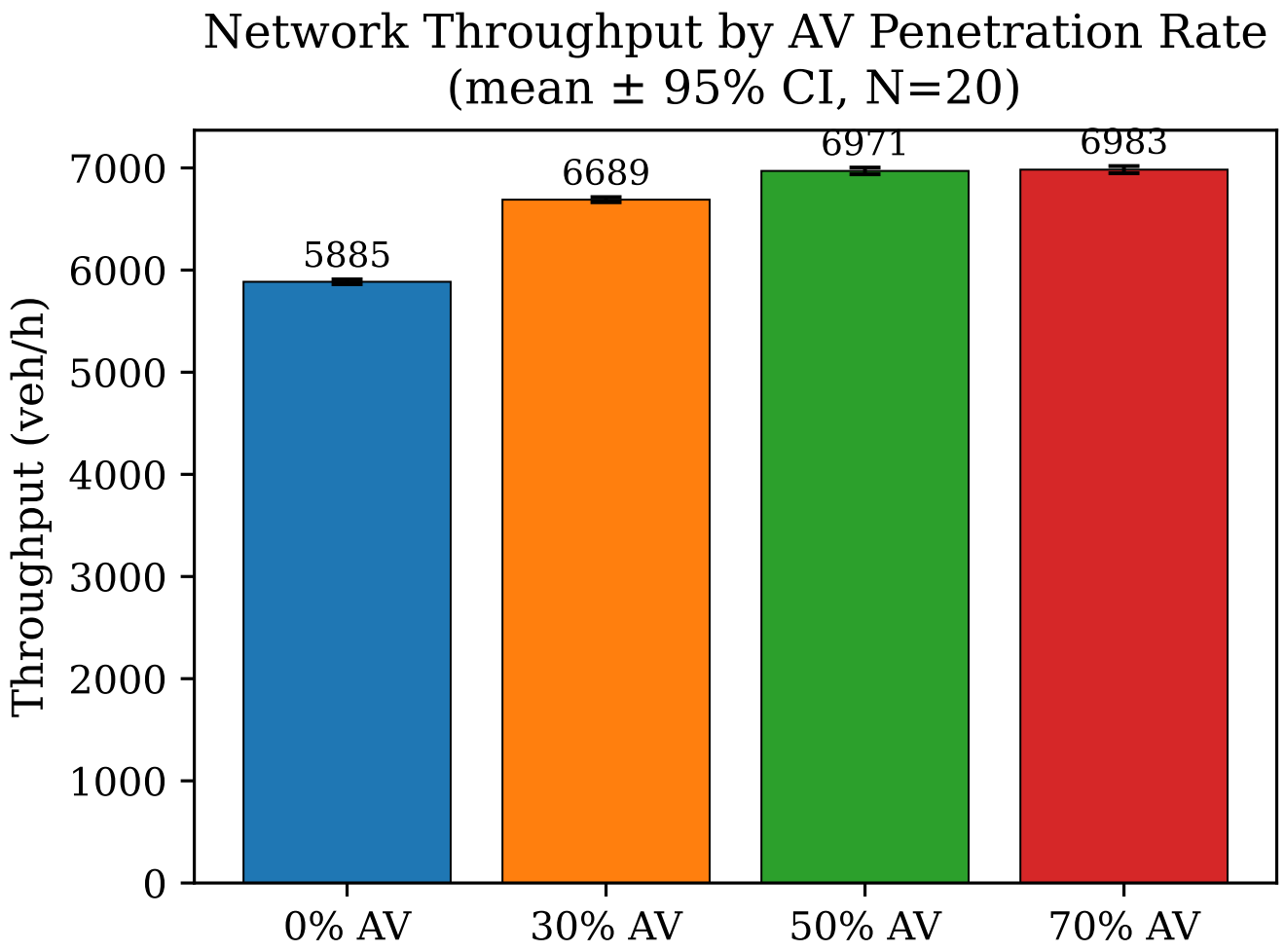


Figure 9: Network throughput across AV penetration scenarios (S1–S4, $N = 20$ replications). Error bars denote 95% confidence intervals. Throughput increases from 5,885 to 6,983 veh/h, saturating at the 7,000 veh/h offered demand.

5.5. Bottleneck Analysis: Lane-Drop Scenario

To evaluate the framework's performance under capacity-reducing geometry, we simulate a 3-lane, 4.5 km freeway segment (600 cells) where the leftmost lane is closed from the midpoint (cell 300) to the end, forcing all lane-3 traffic

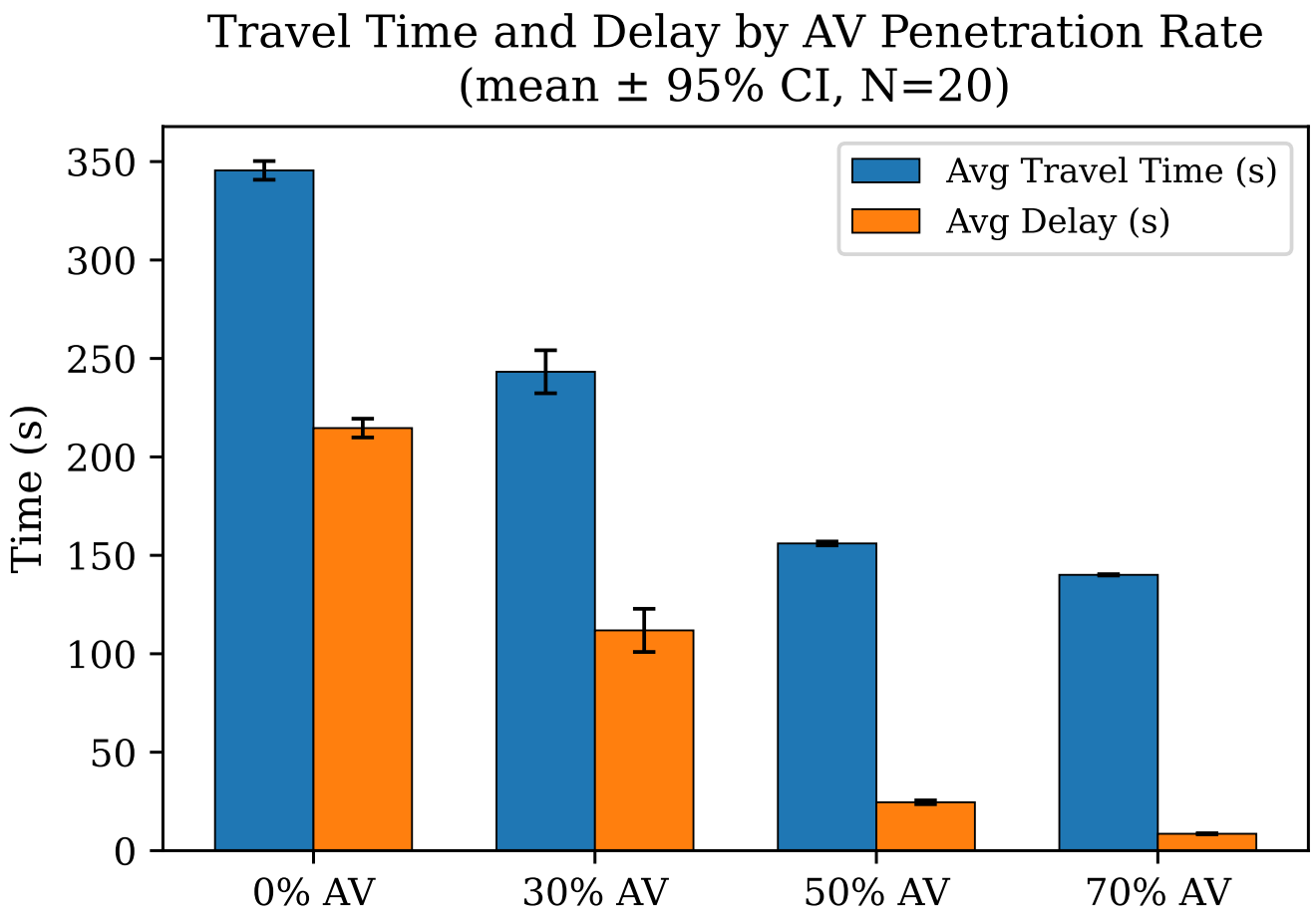


Figure 10: Average travel time across AV penetration scenarios (S1–S4, $N = 20$ replications). Error bars denote 95% confidence intervals. Travel time decreases from 346 s (S1) to 140 s (S4). Trips end at three off-ramps and at the segment end, so the reference is not the full-segment free-flow time of 154 s but the demand-weighted free-flow trip time of 131 s, which S4 exceeds by 7%. The latter is the mean of each origin–destination pair's free-flow time $\ell\,\Delta c/v_f$ weighted by its flow (Section 5.1.3); it is shorter than 154 s because most traffic leaves at off-ramps 1 to 3 rather than driving the full 800 cells.

to merge into lanes 1–2. The closure is a set of blocked cells, the same mechanism used for the incident of Section 5.8, and the segment starts with vehicles placed every 25 cells per lane so the merge is loaded from the first second. Demand is 3,600 veh/h (1,200 veh/h/lane), and AV penetration is varied across 0%, 30%, 50%, and 70%. Each scenario is run with 20 independent replications.

Table 8 summarizes the results.

Throughput at the lane drop. The two-lane throat stops being the constraint between 30% and 50% AV. The lane drop holds the all-HDV stream to 2,510 ± 11 veh/h, that is 70% of the 3,600 veh/h demand: the rest waits upstream. Throughput then rises to 3,135 ± 18 at 30% AV, 3,579 ± 35 at 50% and 3,606 ± 41 at 70%. The last step is 0.8% and the two confidence intervals overlap (Figure 11), but the reason is not that the merge saturates: at 50% the two remaining lanes already carry 99.4% of the offered demand, and at 70% they carry all of it. Between 30% and 50% AV the throat stops being the binding constraint, and beyond that point the demand is.

Delay makes the same transition visible without the ceiling: 257.6±8.4 s at 0% AV, 118.3±11.1 at 30%, 15.7±1.7

Table 8: Bottleneck performance metrics ($N = 20$ replications). Values are means ± 95% confidence intervals. BN_α denotes the bottleneck scenario at AV penetration α.

Metric	$\mathrm{BN}_{0\%}$	$\mathrm{BN}_{30\%}$	$\mathrm{BN}_{50\%}$	$\mathrm{BN}_{70\%}$
Throughput (veh/h)	$2{,}510 \pm 11$	$3{,}135 \pm 18$	$3{,}579 \pm 35$	$3{,}606 \pm 41$
Avg. travel time (s)	372.8 ± 8.4	233.5 ± 11.1	130.9 ± 1.7	120.0 ± 0.2
Avg. delay (s)	257.6 ± 8.4	118.3 ± 11.1	15.7 ± 1.7	4.8 ± 0.2
LC frequency (/veh/km)	0.84 ± 0.01	0.75 ± 0.02	0.40 ± 0.01	0.22 ± 0.00
Vehicles delayed > 20 s (%)	96.9 ± 0.6	92.6 ± 2.6	28.8 ± 6.2	1.2 ± 0.2
Mean cells held	3.95 ± 0.06	4.41 ± 0.16	5.62 ± 0.04	5.00 ± 0.02
Mean origin wait (s)	1.7 ± 0.2	1.0 ± 0.1	0.7 ± 0.1	0.4 ± 0.0
Never entered (veh)	2.1 ± 1.7	0.8 ± 0.5	0.4 ± 0.4	0.2 ± 0.2
Gap rejections ($\times 10^3$)	53.3 ± 1.5	34.0 ± 1.6	14.1 ± 0.8	6.5 ± 0.2

at 50% and 4.8 ± 0.2 at 70%, and the share of vehicles delayed by more than 20 s falls from 96.9% to 1.2%. The 50% and 70% scenarios, indistinguishable in throughput, differ by a factor of three in delay: once every vehicle gets through, what an additional AV buys is time, not flow. This is the concavity of the theoretical capacity function $q_{\max}(\alpha)$ (Eq. 35) meeting a fixed demand, rather than a property of the merge itself.

Run-to-run variability is larger here than on the open freeway, and it is delay, not throughput, that carries it: the coefficient of variation of throughput stays between 0.9% and 2.4% (0.8–1.1% for S1–S4), while for delay it reaches 23% at $\mathrm{BN}_{50\%}$: the 20 per-seed delays spread from 10 to 24 s, with the spread driven by the stochastic order in which HDVs and AVs reach the merge.

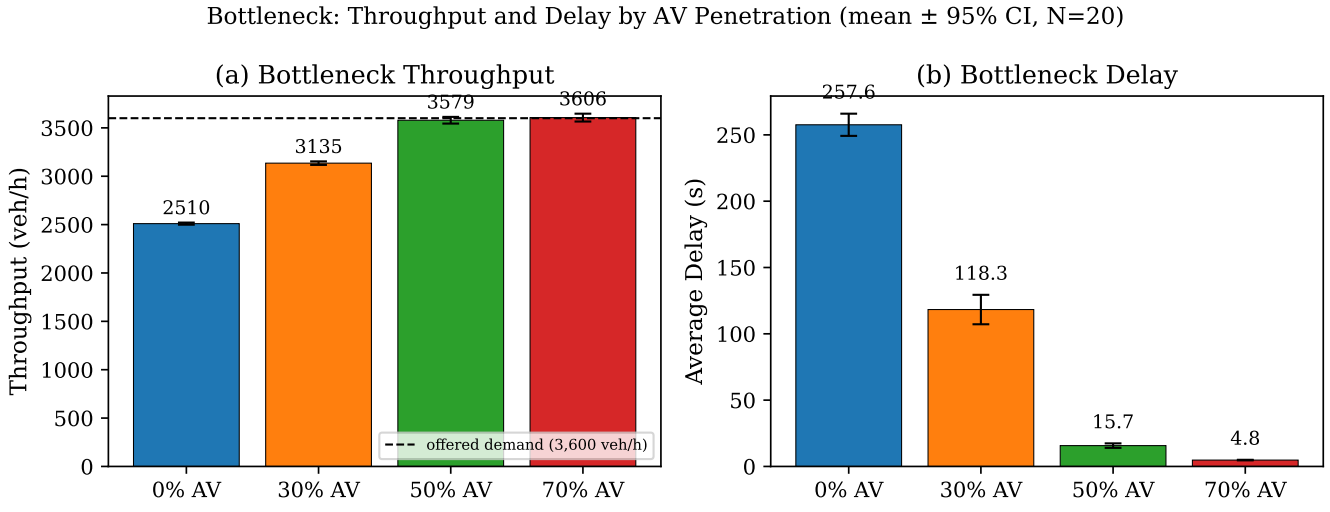


Figure 11: Bottleneck throughput across AV penetration levels ($N = 20$ replications). Error bars denote 95% confidence intervals. Throughput rises from 2,510 to 3,606 veh/h and flattens between 50% and 70% AV, where the two remaining lanes already carry 99% of the 3,600 veh/h demand (dashed line) and, at 70%, all of it; beyond that point the extra capacity shows up as delay reduction, not as flow.

5.6. Sensitivity to HDV Action Interval

The HDV action interval δ_{eval} (the maximum time between driver re-evaluations; Section 3.6) is a key model parameter that controls how frequently human drivers reassess speed and lane-change decisions. The default value of $\delta_{\text{eval}} = 1.0\,\text{s}$ reflects typical human perception-reaction cycling, but lower values approximate more attentive driving or a finer-grained discrete-time approximation of continuous decision-making.

To quantify the sensitivity of simulation outcomes to this parameter, we run the S1 (all-HDV) scenario at three action-interval values: $\delta_{\text{eval}} \in \{0.25, 0.5, 1.0\}\,\text{s}$, with 10 replications at $\delta_{\text{eval}} = 0.25$ and $0.5\,\text{s}$ and 20 replications at $\delta_{\text{eval}} = 1.0\,\text{s}$ (the baseline from Section 5.4). Table 9 reports the results; Figure 12 presents the trends graphically.

Table 9: Sensitivity of S1 (0% AV) performance to HDV action interval. Values are means ± 95% CI.

Metric	$\delta_{\text{eval}} = 0.25\,\text{s}$	$\delta_{\text{eval}} = 0.5\,\text{s}$	$\delta_{\text{eval}} = 1.0\,\text{s}$
Throughput (veh/h)	$6{,}245 \pm 39$	$6{,}173 \pm 32$	$5{,}885 \pm 24$
Avg. travel time (s)	327.2 ± 6.9	328.0 ± 5.5	345.5 ± 4.8
Avg. delay (s)	195.3 ± 6.8	196.4 ± 5.4	214.6 ± 4.8
LC frequency (/veh/km)	2.00 ± 0.01	2.14 ± 0.01	2.32 ± 0.01

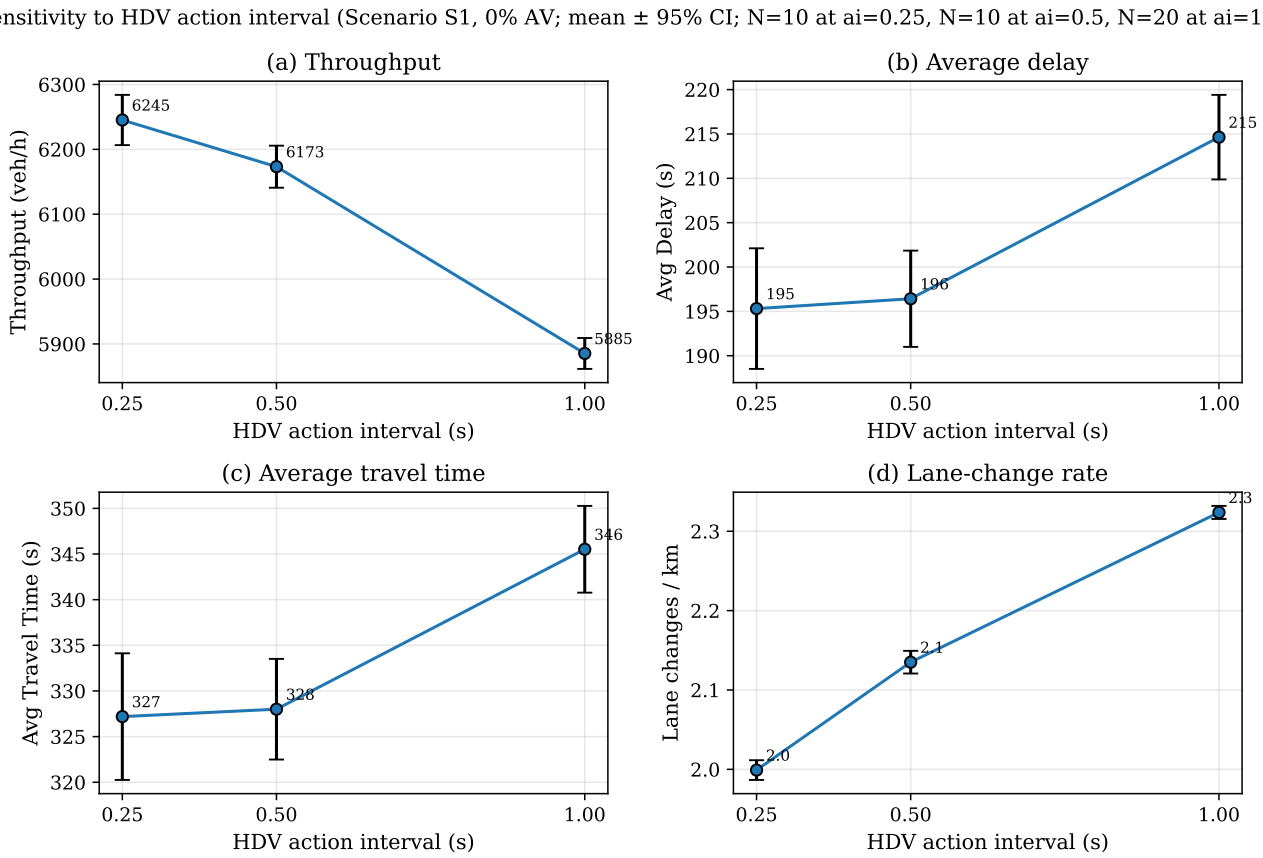


Figure 12: Sensitivity of S1 performance metrics to HDV action interval δ_{eval}. Panels show throughput, average travel time, average delay, and lane-change frequency. Error bars denote 95% confidence intervals. Reducing the action interval monotonically improves all metrics.

Reducing the action interval from 1.0 s to 0.25 s increases throughput by 6.1% (from 5,885±24 to 6,245±39 veh/h) and reduces average delay by 9% (from 215 ± 5 to 195 ± 7 s). Most of that comes from the first halving: 0.5 s already gives $6{,}173 \pm 32$ veh/h, and the step to 0.25 s adds 1.2% more. The effect is monotonic across all metrics: finer-grained driver re-evaluation enables smoother speed adjustments and earlier gap exploitation, reducing the stop-and-go oscillations that arise when drivers react to stale state information. Lane-change frequency decreases at lower action intervals, indicating that more responsive drivers maintain better longitudinal flow, reducing the need for discretionary lane changes.

This result has a structural interpretation: as $\delta_{\text{eval}} \to 0$, the event-driven driver process converges toward the continuous-time limit of the Newell car-following model, closing the gap between the discrete-event implementation and the underlying continuous-time theory. The 6.1% throughput improvement from a 4× reduction in action interval quantifies this structural gap, and its concentration in the first halving suggests that the default 1.0 s value is a conservative but reasonable operating point.

Scope of the sensitivity analysis. This sensitivity analysis is restricted to S1 (all-HDV) because the action interval is an HDV parameter; AVs operate on a fixed 10 Hz controller cycle irrespective of δ_{eval}. In mixed-traffic scenarios (S2–S4), HDVs constitute a decreasing fraction of the fleet, so the sensitivity to δ_{eval} is proportionally smaller. Additionally, reducing δ_{eval} in mixed-traffic scenarios with high event density proved computationally intractable within the experimental budget. Note that the $\delta_{\text{eval}} = 0.25$ and 0.5 s scenarios use $N = 10$ replications versus $N = 20$ for the baseline at $\delta_{\text{eval}} = 1.0$ s; accordingly, the 95% confidence intervals at the two lower values are computed with fewer degrees of freedom and are somewhat wider than they would be at $N = 20$.

5.7. Computational Performance

5.7.1. Scalability with Network Size

To assess how the event-driven architecture scales with network size, we simulate the S1 (all-HDV) scenario on networks of increasing size: 200, 400, 800, 1,600, and 3,200 cells per lane (4 lanes each, yielding 800 to 12,800 total cells), with 3 seeds per configuration. All networks use the same demand density and simulation duration ($T_{\text{sim}} = 1{,}800$ s). Table 10 reports the results; Figure 13 presents the scaling trends.

Table 10: Computational scalability across network sizes (S1, all-HDV, $N = 3$ seeds per size, $T_{\text{sim}} = 1{,}800$ s, means over seeds). Total cells = num_cells × 4 lanes. Demand is the same at every size (3,040 vehicles generated), so a longer network means longer trips rather than more vehicles.

Cells/lane	Total cells	SimPy events ($\times 10^6$)
200	800	6.00
400	1,600	11.35
800	3,200	20.91
1,600	6,400	38.24
3,200	12,800	66.57

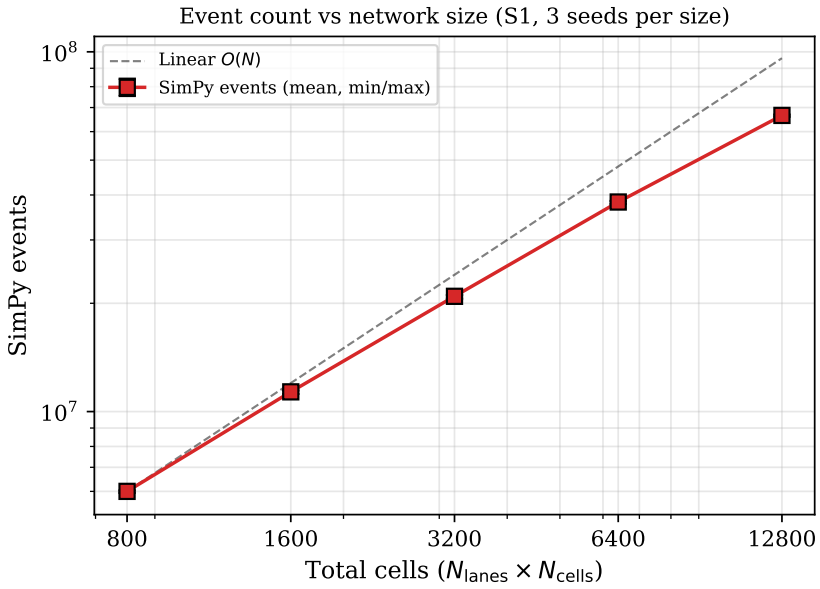


Figure 13: SimPy event count against network size (S1, all-HDV, 3 seeds per size, log–log scale, min to max over seeds as error bars, the dashed line linear growth from the smallest network). The count grows sub-linearly with network length at constant demand (empirical exponent 0.87).

The event count increases by a factor of 11.1 for a 16-fold increase in length (200 to 3,200 cells per lane), an empirical exponent of 0.87. The demand is the same at every size, so this is the cost of longer trips and not of more traffic: each vehicle crosses 16 times as many cells, and each cell crossing costs the same handful of events. The shortfall from 16 is where the event-driven design shows: the short network is congested with the same demand, and contention there adds waits and re-evaluations per cell that the long, free-flowing network does not pay.

5.7.2. Event Count Analysis

To assess the computational characteristics of the event-driven approach on a per-scenario basis, we compare the number of simulation events against a timestep-equivalent simulation on the same 4-lane, 800-cell network.

In a timestep-based simulator with update interval Δt, the number of update operations is:

$$N_{\text{timestep}} = \bar{N}_{\text{vehicles}} \times \frac{T_{\text{sim}}}{\Delta t} \tag{41}$$

where $\bar{N}_{\text{vehicles}}$ is the average number of vehicles simultaneously present in the network. Every vehicle is evaluated at every timestep regardless of its interaction state.

In ODCA-DES the work splits into two kinds, and they should be counted separately.

Movement events implement the resource protocol: per cell traversed, a request, a seize, the traversal timeout (or several of them, when the vehicle is slow enough to be re-timed while crossing) and the delayed release, plus a wait whenever the next cell is held. They have no counterpart in a timestep model, which moves a vehicle by arithmetic on its position, and they dominate the total: the scenarios below execute 38–41 million SimPy events per hour of simulated traffic, almost all of them movement.

Decision events are the behavioural layer, and they are what a timestep model repeats for every vehicle at every step: a speed evaluation (car following when a leader is in range, free flow otherwise) and a direction evaluation

per driver wake-up, or per controller cycle for an AV.

Table 11 compares the event counts across the S1–S4 scenarios.

Table 11: Event counts, one run per scenario ($T_{\text{sim}} = 3600\,\text{s}$, 4 lanes, 800 cells, seed 42). SimPy events are every event the calendar processed; speed evaluations are the driver decisions. The timestep row is the baseline of Eq. (41), the average number of vehicles in the network times $T_{\text{sim}}/\Delta t$ at $\Delta t = 0.25\,\text{s}$.

Metric	S1 (0%)	S2 (30%)	S3 (50%)	S4 (70%)
SimPy events ($\times 10^6$)	40.1	40.6	39.3	38.6
Speed evaluations ($\times 10^6$)	2.22	5.72	5.95	7.04
of which car following ($\times 10^6$)	1.91	5.06	4.79	5.25
Timestep updates ($\times 10^6$)	7.40	5.96	4.36	3.93
Lane changes	69,115	59,068	34,274	19,036
Average vehicles in network	514	414	303	273

For the all-HDV baseline (S1), the drivers make 2.22 million speed evaluations, of which 1.91 million involve a leader. A timestep model with $\Delta t = 0.25\,\text{s}$ would evaluate the same fleet 7.40 million times, given the 514 vehicles present on average (throughput times mean travel time): the event-driven driver process does 3.3 times less behavioural work for the same traffic, because a vehicle with no neighbour moving near it is not woken. That ratio narrows as AVs enter, and reverses: the controller decides for every AV ten times a second whether anything has changed or not, so at 70% AV the model makes 7.04 million evaluations against 3.93 million timestep updates, 1.8 times more. The saving is a property of the event-driven driver, not of the framework, and a fixed-rate controller gives it away. Lane changes fall from 69,115 to 19,036 with AV penetration, since AVs make only mandatory ones.

What the counts show is the *locality* of events: each event involves only one vehicle and its immediate neighbors, and there is no global synchronization step. The event-driven driver process produces it: a vehicle in free flow with no neighbor near it generates only periodic re-evaluations, so the behavioural work concentrates at congested locations where interactions occur and is proportional to the *interacting* vehicle count rather than the total.

5.8. Dynamic Congestion Analysis

To demonstrate the framework's ability to capture transient traffic dynamics, we simulate a temporary incident on a 4-lane, 4.5 km segment (600 cells, no ramps). The segment is initialized with vehicles at free-flow density (one vehicle every 30 cells per lane, ≈4.4 veh/km/lane) to establish stable initial conditions, followed by a 200 s warm-up period. That is shorter than the 300 s used for S1–S4 because the road starts populated and is shorter (4.5 km rather than 6 km), so a vehicle entering at $t = 0$ has left before the closure begins; the measurement here is the transient itself, timed from the closure, not a steady-state mean. Mainline demand is 4,500 veh/h (1,125 veh/h/lane), distributed equally across all lanes, which the open four-lane segment carries in free flow (mean delay 12.9 s over the hour). At $t = 300\,\text{s}$ the two leftmost lanes (3 and 4) are blocked over cells 250–260 (11 cells, 82.5 m incident zone), as a crash blocking half the road would, and the blockage is removed at $t = 1500\,\text{s}$, a 20-minute incident. Two lanes must then carry a demand that needs about three, so the closure is binding: this creates the three phases

of interest, free flow, queue growth during the closure, and recovery after reopening. Over the hour the incident run serves 4,228 veh/h against the baseline's 4,536, and its mean delay is 245.7 s against 12.9 s.

Figure 14 presents lane-by-lane time-space diagrams comparing the baseline (left) and incident (right) scenarios. In the baseline, trajectories are uniformly steep across all lanes throughout the simulation, confirming free flow at 4,500 veh/h. Upon closure, lanes 3 and 4 empty downstream of the incident zone while their traffic merges right: mandatory lane changes, triggered early by the blockage-aware urgency inflation (Section 3.6.2), move it into lanes 1 and 2, which then carry the whole demand.

The queue grows steadily from the closure at $t = 300$ s, extending upstream at about 0.19 km every one to two minutes, and reaches 1.88 km at $t = 1{,}080$ s. That 1.88 km is the full distance from the segment entry to the blocked cells, so from that point the queue length is censored by the segment boundary: vehicles that cannot enter appear as origin wait rather than as additional queue. Speed in the kilometre upstream of the blocked cells (Edie's definition, as for the fundamental diagram) falls from the baseline's 4.8–5.1 cells/s to about 1.0 cells/s during the closure, a fifth of free flow. The queue still fills the segment at reopening ($t = 1{,}500$ s), stays at the entry until about $t = 2{,}160$ s and is gone by $t = 2{,}280$ s. Upstream speeds return to within 2% of the baseline at $t = 2{,}460$ s, that is 16 minutes after the lanes reopened for a 20-minute closure. This is a single run (seed 42), and these figures are read from the trajectories of the baseline and incident runs. Nothing in the model schedules any of this; the queue is vehicles waiting for cells that are not released, and the recovery is the same mechanism running in reverse.

Figure 15 provides the corresponding lane-by-lane density heatmaps, mirroring the trajectory layout. The baseline columns (left) show uniform density throughout all lanes, confirming free-flow conditions at the given demand level. The incident columns (right) reveal the congestion dynamics per lane: lanes 3 and 4 hold the jam that forms against the blocked cells and are empty downstream of them, while lanes 1 and 2 take the merging traffic and carry the queue upstream with them. Within the queue, diagonal bands of high density run backwards in time-space, the stop-and-go waves of a congested stream; the boundary between the congested and free-flowing regions is sharp, as kinematic wave theory predicts, and it emerges here from cells being released one reaction time after their holder moves on, with no macroscopic flow model anywhere in the simulation.

After clearance the picture inverts: the bands tilt the other way, forwards, as the platoons discharged from the queue travel downstream at free-flow speed and spread out over the remaining kilometres of the segment. Upstream conditions return to the baseline 16 minutes after the lanes reopen. The queue discharges more slowly than it formed, the reduced discharge capacity that empirical studies of active freeway bottlenecks report [37].

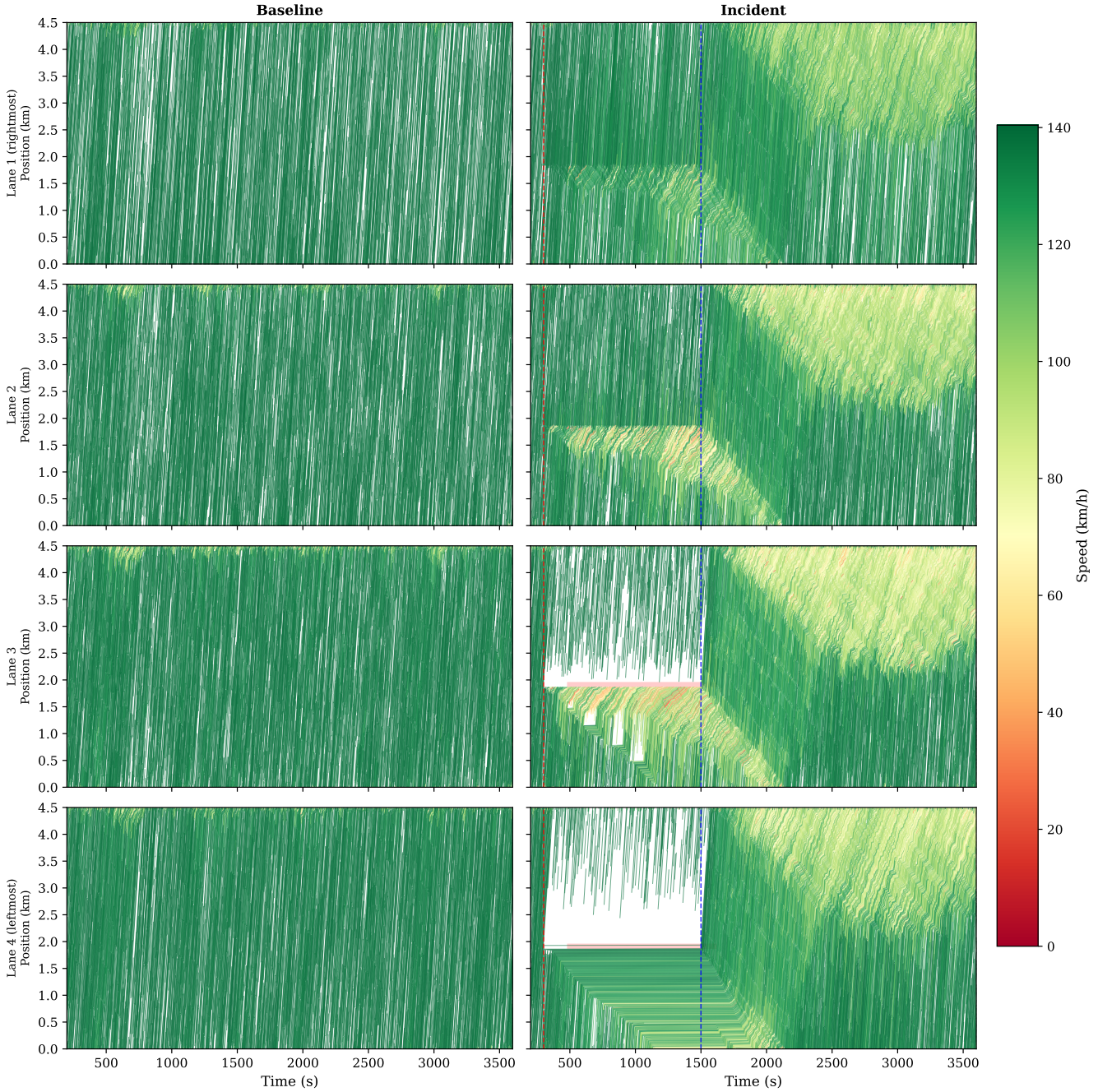


Figure 14: Lane-by-lane time-space diagrams: baseline (left) vs. the 20-minute closure of lanes 3 and 4 (right), at 4,500 veh/h. Trajectories are coloured by speed (green = free flow, red = stopped). Dashed lines on the incident panels mark onset ($t = 300$ s, red) and clearance ($t = 1500$ s, blue), and the shaded band marks the blocked cells. The baseline is free-flowing throughout; the incident panels show the blocked lanes emptying downstream, the queue growing upstream in all four lanes, and the recovery spreading upstream after clearance.

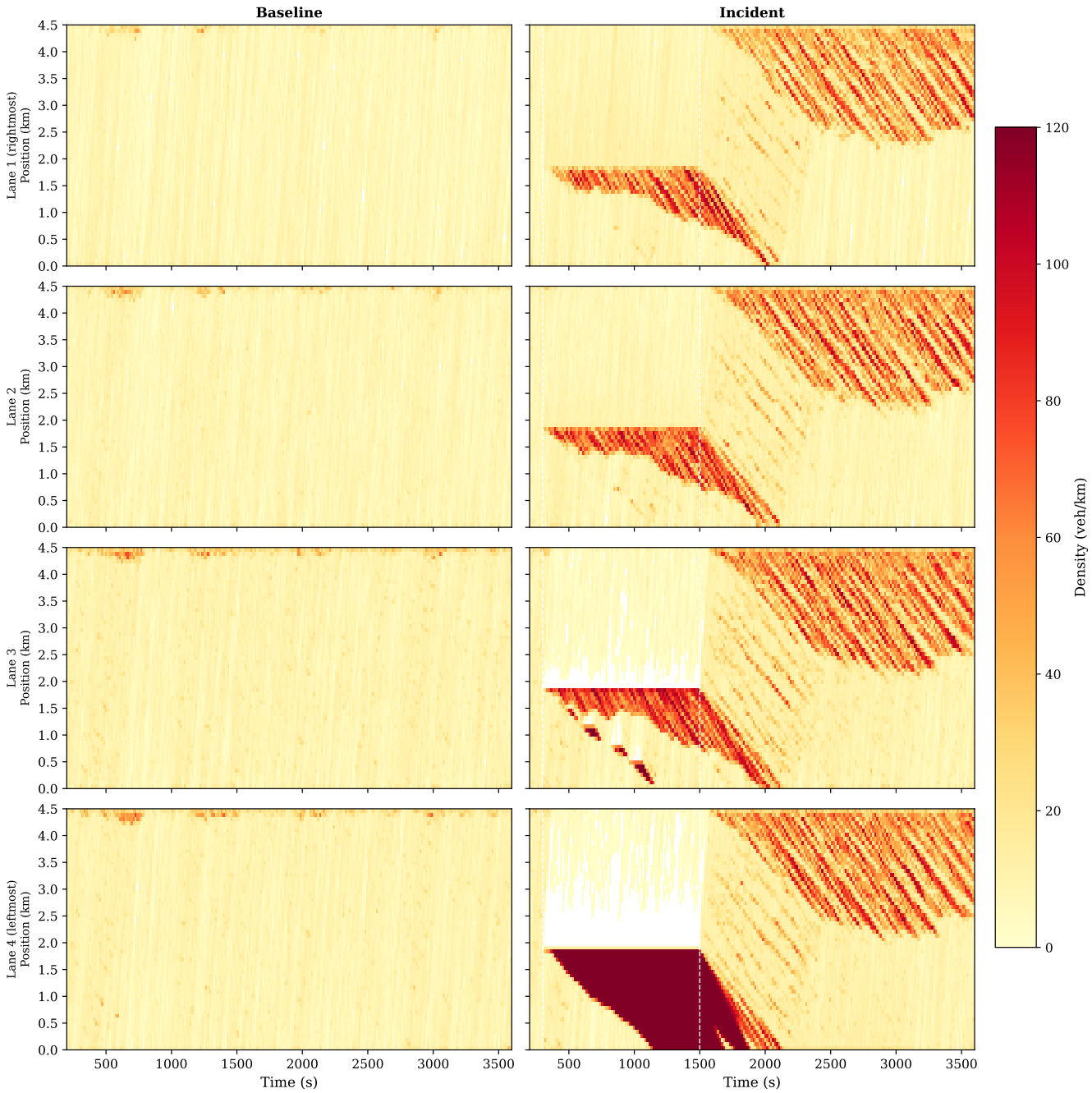


Figure 15: Lane-by-lane time-space density heatmaps (15 s × 75 m bins): baseline (left) vs. the 20-minute closure of lanes 3 and 4 (right). White dashed lines on the incident panels mark incident onset ($t = 300$ s) and clearance ($t = 1500$ s). The uniform baseline contrasts with lane-specific congestion patterns: lane 4 shows direct blockage impact, while lanes 1–3 show progressive density increases from merging traffic.

6. Conclusion

This paper presented the Object-Driven Cellular Automata (ODCA) framework within a Discrete Event Simulation environment for modeling multi-lane freeway traffic with heterogeneous vehicle populations. The key contributions are:

i) An inversion of the usual assignment of behaviour in a cell-based simulation. Cellular automata have been placed on discrete-event engines before, Cell-DEVS [1, 2] since 2001 and in traffic [3, 4], and that combination is not claimed here. What is new is that the cell holds state (its occupancy, a blocked flag, a posted speed limit) but no transition function, while the vehicle is the process and carries all the behaviour; the lattice becomes a contended medium rather than a field of local rules. The synchronous-update and integer-speed limitations of classical CA do not survive the inversion, because neither the global clock nor the discrete speed state has anything left to attach to.

ii) A resource-based movement protocol (request–wait–seize–delay–release) in which traffic congestion emerges naturally from cell resource contention, without explicit gap-checking or collision-avoidance rules.

iii) A delayed cell-release mechanism that produces headways consistent with Newell's simplified car-following model, establishing a direct analytical link between the simulation mechanics and traffic flow theory.

iv) A hybrid discretization scheme with discrete spatial cells, continuous vehicle speeds, and continuous event-driven time, yielding smooth fundamental diagrams and realistic trajectory outputs natively in the $T(x, n)$ passage-time coordinate system, a representation rarely exploited in simulation.

v) An event-driven driver process where vehicles re-evaluate speed and lane-change decisions in response to neighbor movements rather than at fixed intervals, concentrating computational effort at locations where interactions occur and enabling immediate response to local density changes.

vi) Physically motivated lane-change safety assessment using speed-dependent closing-speed gaps and blockage-aware mandatory lane-change urgency inflation, producing realistic incident dynamics without requiring cooperative communication between vehicles.

vii) A decoupled architecture separating the driver process (speed and direction decisions) from the movement process (resource acquisition), enabling modular integration of heterogeneous behavioral models for mixed AV/HDV populations.

viii) A closed-form expression for the capacity effect of AV penetration (Eq. 35), derived directly from the resource protocol parameters, providing an analytical basis for assessing AV deployment scenarios.

The analytical results established that the ODCA-DES framework produces a triangular fundamental diagram identical to that of Newell's kinematic wave model, with free-flow speed v_f and backward wave speed $w = \ell/\tau = d/\tau$ emerging directly from the cell length and delayed-release parameters, the cell length being the jam spacing d of Newell's model. The illustrative comparison with the NaSch model demonstrated that the event-driven, continuous-speed approach eliminates the integer-speed artifacts and artificial synchronization inherent in classical CA, while the resource-based conflict resolution provides a physically consistent and deadlock-free alternative to rule-based gap checking.

The event-driven driver process differs fundamentally from both classical CA and fixed-timestep microsimulators: vehicles re-evaluate their behavior in response to neighbor movements rather than at predetermined intervals, naturally producing adaptive evaluation frequencies that match the local interaction intensity. In free flow, a vehicle generates only periodic re-evaluations; in congestion, the same vehicle responds immediately to every neighbor movement. This push-based architecture, combined with speed-dependent closing-speed safety gaps and blockage-aware lane-change urgency inflation, produces physically realistic incident dynamics (including upstream spillback and post-clearance recovery) without requiring cooperative communication or centralized coordination among human-driven vehicles.

Computational experiments with 20 independent replications demonstrated the framework's ability to reproduce expected traffic dynamics across multiple scenarios. The fundamental diagram validation confirmed close agreement with the theoretical triangular FD: on a ring road, which removes the entry boundary effect, the simulated capacity reaches 92 to 96% of the analytic $q_{\max}$, with scatter patterns arising from per-driver heterogeneity. Under increasing AV penetration from 0% to 70%, served throughput increased by 18.7% (from $5{,}885 \pm 24$ to $6{,}983 \pm 35$ veh/h) and average delay decreased by 96% (from 215 ± 5 to 8.6 ± 0.3 s) on a congested 4-lane network with per-lane origin–destination flows. Both experiments show the same transition: once enough AVs are present for the network to serve the demand offered to it (99.8% of 7,000 veh/h at 70% AV on the open freeway; 99.4% of 3,600 veh/h from 50% AV at the lane drop and all of it at 70%), throughput cannot record further capacity, and the benefit appears in delay, which falls by a further factor of three between the 50% and 70% bottleneck scenarios (15.7 ± 1.7 to 4.8 ± 0.2 s) whose throughputs are statistically indistinguishable. Sensitivity analysis showed that reducing the HDV action interval from 1.0 s to 0.25 s increases throughput by 6.1%, most of it in the first halving, quantifying the structural gap between the discrete-event implementation and the continuous-time theoretical limit. The temporary incident experiment showed that backward-propagating stop-and-go waves, congestion spillback, and gradual recovery emerge naturally from the event-driven resource protocol.

Across network sizes from 800 to 12,800 cells, the event count grew by 11.1 for a 16-fold increase in trip length, an empirical exponent of 0.87. The DES paradigm's event locality, where computational effort concentrates at congested locations and times, is a property fixed-timestep approaches cannot have without significant algorithmic modification. Nothing in the request–wait–seize–delay–release protocol, in the cell without rules, or in the event-driven re-evaluation is specific to vehicles: the pattern is a lattice of unit-capacity resources acquired by object processes, and it is in principle transferable wherever discrete space is a contended medium. That is a structural observation about the protocol rather than a demonstrated result, and one domain is reported here. Traffic is the case in which the pattern can be validated against a closed-form theory, which is why it is the case chosen.

6.1. Limitations

Several limitations of the current framework should be noted:

- **No empirical calibration.** All model parameters (reaction times, headway distributions, lane-change thresholds) are derived from representative values in the literature rather than calibrated to a specific field dataset. Validation against empirical trajectory data such as NGSIM or highD would require matching simulated and observed fundamental diagrams, headway distributions, and lane-change rates at specific sites. Until such

calibration is performed, the quantitative predictions (absolute throughput, delay magnitudes) should be interpreted as framework demonstrations rather than site-specific forecasts.

- **Single unidirectional freeway segment.** The test network is a single straight freeway with on/off-ramps. There are no merge/diverge junctions, weaving sections, signalized ramp meters, or route-choice decisions. The resource-based architecture is in principle extensible to such geometries, but the current implementation does not support them.

- **Static demand.** All experiments use constant origin–destination flow rates. The event-driven architecture is compatible with time-varying demand (injection rates can be scheduled as time-dependent events), but the current implementation does not expose this capability. Consequently, the framework has not been tested under peak-hour demand ramps, demand shocks, or diurnal patterns.

- **Limited heterogeneity calibration.** While per-driver parameter sampling (LogNormal for τ and action interval, Normal for slowdown probability) introduces within-type heterogeneity, the distribution parameters are literature-derived rather than fitted to observed driver populations. Lane-change parameters remain homogeneous within each vehicle type.

- **Sensitivity scope.** The action-interval sensitivity analysis (Section 5.6) covers only the all-HDV scenario (S1). Because the action interval is an HDV parameter and AVs operate on a fixed 10 Hz controller cycle, the sensitivity is greatest in HDV-dominated traffic. At higher AV penetration levels, HDV behavioral parameters have proportionally less influence on aggregate performance. Extending the sensitivity analysis to mixed-traffic scenarios at low action intervals was computationally intractable within the experimental budget.

- **First-order car following.** Speed is a function of spacing with no explicit acceleration or deceleration bound (Section 4.3). Extending the model to second order is outside the scope of this paper.

- **No creeping during lane-change waits.** Vehicles already creep behind a leader that is still moving (Eq. (13)); a vehicle waiting for a lane change does not. When a vehicle's lane-change request cannot be immediately satisfied (target lane cell occupied), the vehicle stops in its current cell and waits. In reality, drivers "creep" forward, inching into gaps incrementally. Modeling creeping requires partial cell occupancy or multi-cell resource acquisition, which is a natural extension of the resource framework.

- **Prototype speed.** The implementation is Python on SimPy and is not tuned for speed, so the paper reports event counts and no run times. A compiled engine with the same event structure is future work.

6.2. Future Research Directions

The ODCA-DES framework provides a foundation for several research directions:

- **Empirical calibration and validation.** The $T(x, n)$ trajectory output format is well-suited for direct comparison with trajectory datasets such as NGSIM and highD, which provide vehicle-level passage times at discrete cross-sections. A systematic calibration study would fit the reaction-time, headway, and lane-change

distributions to observed trajectory data at specific sites, enabling quantitative validation of the fundamental diagram shape, headway distributions, and congestion wave speeds.

- **Network-level extension.** The cell-based resource model can be extended to freeway networks by defining merge and diverge cells with multi-input or multi-output resource connections, enabling corridor-level simulations with interchanges, weaving sections, and route-choice decisions.
- **Time-varying demand.** The event-driven architecture is compatible with time-varying injection rates scheduled as time-dependent events. Implementing and validating this capability would enable simulation of peak-hour demand ramps, demand shocks, and diurnal traffic patterns.
- **Expanded sensitivity analysis.** Extending the action-interval sensitivity to mixed-traffic scenarios (S2–S4) would quantify the interaction between HDV behavioral granularity and AV penetration level. Computational advances (e.g., event batching or parallel DES engines) may be needed to make low-action-interval mixed-traffic runs tractable.
- **Calibrated driver heterogeneity.** Calibrating the per-driver parameter distributions against empirical driver-behavior data would produce more realistic traffic dynamics and enable the study of behavioral variability effects on traffic stability and capacity.
- **Creeping during lane-change waits.** Allowing vehicles to partially advance during lane-change waits would improve realism and throughput prediction, particularly at high-demand weaving sections.
- **Fewer events in free flow.** In less dense traffic a vehicle with no neighbour near it does not need to seize, cross and release every cell one at a time: it could acquire a stretch of free cells at once and be re-evaluated less often, so the interval between its evaluations can grow with the free road ahead of it. This would cut the movement events that dominate the event count (Section 5.7.2).
- **Transfer to a second contended-space domain.** The resource protocol is stated independently of vehicles, but only one domain is reported here. Applying it to another discrete-space system in which occupancy is exclusive and waiting is the congestion mechanism, such as block-signalled rail or automated warehouse aisles, would test whether the analytical tractability shown in Section 4 is a property of the protocol or of the traffic case.
- **Advanced AV coordination.** The decoupled architecture enables integration of cooperative AV behaviors (platoon formation, coordinated lane changes, V2V communication) as controller modules, without modifying the underlying movement engine.

CRediT authorship contribution statement

Kerem Demirtaş: Conceptualization, Methodology, Software, Validation, Formal analysis, Writing (original draft). **Pitu Mirchandani:** Supervision, Writing (review and editing). **Xuesong Zhou:** Supervision, Writing (review and editing).

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The simulator `odca-des` [38], under the MIT licence, and the scripts, configurations and seeds that produce every result, table and figure in this paper are publicly available at `https://github.com/kdemirtas/odca-des` and `https://github.com/kdemirtas/paper-odca-des`, and archived on Zenodo at the versions used here: `https://doi.org/10.5281/zenodo.23923633` (`odca-des` 0.2.0) and `https://doi.org/10.5281/zenodo.23924708` (the release `v2.0-submission` of the paper repository: scripts and configurations). A preprint of this manuscript is archived at `https://doi.org/10.5281/zenodo.23922674`.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) to assist with the simulation code, to check the simulation outputs and the numbers quoted in the manuscript against them, and to review the manuscript for consistency and style. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

[1] G. A. Wainer, N. Giambiasi, Timed Cell-DEVS: Modeling and simulation of cell spaces, in: Discrete Event Modeling and Simulation Technologies, Springer, New York, 2001, pp. 187–214. doi:10.1007/978-1-4757-3554-3_10.

[2] G. A. Wainer, Cellular modeling with Cell-DEVS: A discrete-event cellular automata formalism, in: Cellular Automata, Springer International Publishing, 2014, pp. 6–15. doi:10.1007/978-3-319-11520-7_2.

[3] A. Davidson, G. A. Wainer, Specifying truck movement in traffic models using Cell-DEVS, in: Proceedings of the 33rd Annual Simulation Symposium (SS 2000), IEEE Computer Society, 2000, pp. 66–73. doi:10.1109/simsym.2000.844902.

[4] G. A. Wainer, ATLAS: A language to specify traffic models using Cell-DEVS, Simulation Modelling Practice and Theory 14 (2006) 313–337. doi:10.1016/j.simpat.2005.07.002.

[5] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, E. Wießner, Microscopic traffic simulation using SUMO, in: Proceedings of the IEEE International Conference on Intelligent Transportation Systems, 2018, pp. 2575–2582.

[6] M. Fellendorf, P. Vortisch, Microscopic traffic flow simulator VISSIM, in: Fundamentals of Traffic Simulation, Springer, 2010, pp. 63–93.

[7] K. Nagel, M. Schreckenberg, A cellular automaton model for freeway traffic, Journal de Physique I 2 (1992) 2221–2229.

[8] S. Maerivoet, B. De Moor, Cellular automata models of road traffic, Physics Reports 419 (2005) 1–64.

[9] G. F. Newell, A simplified theory of kinematic waves in highway traffic, part I: General theory, Transportation Research Part B: Methodological 27 (1993) 281–287.

[10] M. Rickert, K. Nagel, M. Schreckenberg, A. Latour, Two lane traffic simulations using cellular automata, Physica A: Statistical Mechanics and its Applications 231 (1996) 534–550.

[11] W. Knospe, L. Santen, A. Schadschneider, M. Schreckenberg, Towards a realistic microscopic description of highway traffic, Journal of Physics A: Mathematical and General 33 (2000) L477–L485. doi:10.1088/0305-4470/33/48/103.

[12] M. Treiber, A. Hennecke, D. Helbing, Congested traffic states in empirical observations and microscopic simulations, Physical Review E 62 (2000) 1805–1824.

[13] P. G. Gipps, A behavioural car-following model for computer simulation, Transportation Research Part B: Methodological 15 (1981) 105–111.

[14] A. Kesting, M. Treiber, D. Helbing, General lane-changing model MOBIL for car-following models, Transportation Research Record 1999 (2007) 86–94.

[15] J. Banks, J. S. Carson, B. L. Nelson, D. M. Nicol, Discrete-Event System Simulation, 5th ed., Pearson, 2010.

[16] W. Burghout, H. N. Koutsopoulos, I. Andréasson, Hybrid mesoscopic–microscopic traffic simulation, Transportation Research Record 1934 (2005) 218–225.

[17] J. Barceló, Models, traffic models, simulation, and traffic simulation, in: Fundamentals of Traffic Simulation, Springer, 2010, pp. 1–62. doi:10.1007/978-1-4419-6142-6_1.

[18] C. Osorio, M. Bierlaire, A simulation-based optimization framework for urban transportation problems, Operations Research 61 (2013) 1333–1345. doi:10.1287/opre.2013.1226.

[19] B. P. Zeigler, H. Praehofer, T. G. Kim, Theory of Modeling and Simulation: Integrating Discrete Event and Continuous Complex Dynamic Systems, 2 ed., Academic Press, San Diego, CA, 2000.

[20] SimPy Development Team, SimPy: Discrete event simulation for Python, 2023. URL: https://simpy.readthedocs.io/.

[21] M. Bando, K. Hasebe, A. Nakayama, A. Shibata, Y. Sugiyama, Dynamical model of traffic congestion and numerical simulation, Physical Review E 51 (1995) 1035–1042.

[22] G. F. Newell, A simplified car-following theory: a lower order model, Transportation Research Part B: Methodological 36 (2002) 195–205.

[23] Z. Zheng, Recent developments and research needs in modeling lane changing, Transportation Research Part B: Methodological 60 (2014) 16–32.

[24] P. G. Gipps, A model for the structure of lane-changing decisions, Transportation Research Part B: Methodological 20 (1986) 403–414.

[25] T. Toledo, H. N. Koutsopoulos, M. Ben-Akiva, Modeling integrated lane-changing behavior, Transportation Research Record 1857 (2003) 30–38.

[26] K. Demirtaş, P. Mirchandani, X. Zhou, Probabilistic lane changing models based on logistic functions, Preprint, Zenodo, 2026. doi:10.5281/zenodo.23922672.

[27] A. Talebpour, H. S. Mahmassani, Influence of connected and autonomous vehicles on traffic flow stability and throughput, Transportation Research Part C: Emerging Technologies 71 (2016) 143–163.

[28] S. E. Shladover, D. Su, X.-Y. Lu, Impacts of cooperative adaptive cruise control on freeway traffic flow, Transportation Research Record 2324 (2012) 63–70.

[29] B. van Arem, C. J. van Driel, R. Visser, The impact of cooperative adaptive cruise control on traffic-flow characteristics, IEEE Transactions on Intelligent Transportation Systems 7 (2006) 429–436.

[30] M. J. Lighthill, G. B. Whitham, On kinematic waves. II. A theory of traffic flow on long crowded roads, Proceedings of the Royal Society of London. Series A 229 (1955) 317–345.

[31] P. I. Richards, Shock waves on the highway, Operations Research 4 (1956) 42–51.

[32] C. F. Daganzo, A variational formulation of kinematic waves: basic theory and complex boundary conditions, Transportation Research Part B: Methodological 39 (2005) 187–196.

[33] B. D. Greenshields, A study of traffic capacity, Highway Research Board Proceedings 14 (1935) 448–477.

[34] C. F. Daganzo, The cell transmission model: A dynamic representation of highway traffic consistent with the hydrodynamic theory, Transportation Research Part B: Methodological 28 (1994) 269–287.

[35] L. C. Edie, Discussion of traffic stream measurements and definitions, Proceedings of the 2nd International Symposium on the Theory of Traffic Flow (1963) 139–154.

[36] M. J. Cassidy, Bivariate relations in nearly stationary highway traffic, Transportation Research Part B: Methodological 32 (1998) 49–59.

[37] M. J. Cassidy, R. L. Bertini, Some traffic features at freeway bottlenecks, Transportation Research Part B: Methodological 33 (1999) 25–42. doi:10.1016/S0191-2615(98)00023-X.

[38] K. Demirtaş, odca-des: Object-driven cellular automata discrete event traffic simulator, [software] Zenodo, https://github.com/kdemirtas/odca-des, 2026. doi:10.5281/zenodo.23923633, MIT licence.